L’intelligence artificielle dans les réseaux informatiques

SIEM, gestion des données et cas d’usage

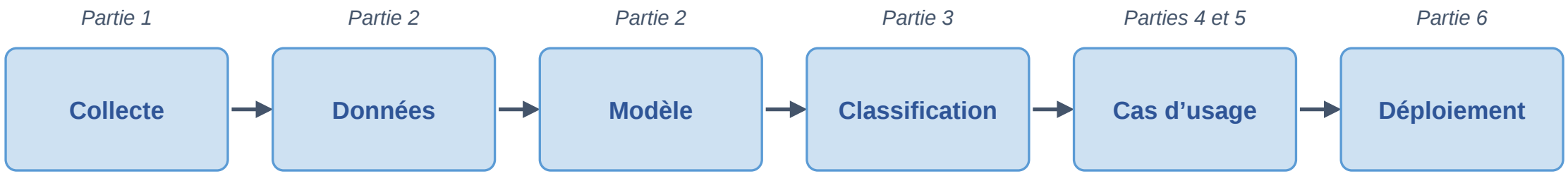
Université Claude Bernard Lyon 1

Objectifs du cours

1. **Collecter** : choisir où et comment observer un réseau (TAP, eBPF, flux, télémétrie, logs, SIEM)
2. **Modéliser** : relier entrées, sorties et familles de modèles, et évaluer sans se tromper
3. **Classifier** : réduire le volume dès la collecte en ne gardant que l'utile
4. **Appliquer** : détecter des anomalies et inférer la qualité d'expérience vidéo

Plan du cours

1. **Collecte** : TAP, eBPF, flux, logs, SIEM
2. **L'IA** : entrées, sorties, modèles, évaluation
3. **Classification** : réduire le volume dès la collecte
4. **Détection** : règles, ML, UEBA, SIEM, SOAR
5. **Performances** : inférence de QoE, cas Netflix



Introduction

Ce que l’IA apporte à l’exploitation d’un réseau

Besoin	Aujourd’hui (sans IA)	Avec l’IA	Remplace ou complète ?
Détecter les attaques	Signatures et règles écrites à la main	Détection de comportements jamais vus	Complète les règles
Superviser les performances	Seuils fixes sur les compteurs	Seuils adaptatifs, prévision de la charge	Remplace les seuils manuels
Classifier le trafic	Numéros de port, inspection du contenu	Classification sur les métadonnées, même chiffré	Remplace l’inspection du contenu
Estimer la qualité vécue	Mesures de QoS (débit, latence)	Inférence de la QoE de l’abonné	Complète la QoS
Répondre aux incidents	Diagnostic et réponse manuels	Aide à la décision, automatisation	Complète l’opérateur

L’IA ne remplace pas l’opérateur ni les protections de base (pare-feu, mises à jour) : elle les complète.

Trois raisons d’utiliser l’IA dans les réseaux

- **Volume** : liens à 10, 40, 100 Gb/s : des millions de paquets par seconde, impossibles à inspecter à la main.
- **Complexité** : réseaux multi-constructeurs, cloud, IoT, SDN : trop d’interactions pour des règles écrites une à une.
- **Opacité** : le trafic est massivement chiffré : on ne voit plus le contenu, il faut apprendre à partir des métadonnées.

Les outils d’exploitation réseau et leurs limites sans IA

Outil	À quoi il sert	Données traitées	Limites sans IA
Supervision (Zabbix, Nagios, Prometheus)	Surveiller la disponibilité et les performances	SNMP, flux, métriques	Seuils fixes réglés à la main ; pas de prévision
IDS / IPS : détection et prévention d’intrusion (Snort, Suricata)	Détecter ou bloquer des attaques connues	Paquets	Aveugle aux attaques nouvelles ; signatures à maintenir ; peu utile sur trafic chiffré
NDR : Network Detection and Response (Darktrace, Vectra)	Détecter des menaces dans le trafic réseau	Flux, paquets	Règles de comportement écrites à la main
SIEM : Security Information and Event Management (Splunk, Elastic, Wazuh)	Centraliser et corréler les logs de tout le SI	Logs, événements	Règles de corrélation à écrire ; déluge d’alertes
SOAR : Security Orchestration, Automation and Response (Cortex XSOAR, Shuffle)	Automatiser la réponse aux alertes	Alertes du SIEM	Scénarios figés : ne traite que ce qui a été prévu

Ces outils sont ceux d’un opérateur ou d’un SOC (Security Operations Center, l’équipe d’analystes).

Les limites des approches classiques

- **Seuils manuels** : « Alerte si le débit dépasse X ». Ignore la saisonnalité, génère du bruit, se règle lien par lien.
- **Signatures** : détectent ce qui est déjà connu (IDS : Snort, Suricata). Aveugles aux attaques nouvelles ou obfusquées.
- **Règles statiques** : corrélation écrite par des experts. Lisible et auditable, mais coûteuse à maintenir et facile à contourner.

Ce que le ML apporte

Règles expertes

- Connaissance codée à la main
- Comportement prévisible et explicable
- Couverture limitée à ce qu'on a anticipé
- Maintenance continue

Apprentissage automatique

- Régularités apprises sur les données
- Détecte des motifs que personne n'a écrits
- Dépend de la qualité des données et des labels
- Peut dériver : à surveiller et à réentraîner

Le ML ne remplace pas les règles : il complète ce qu'elles ne couvrent pas.

Quand ne pas utiliser le ML

- **Peu de données** ou aucune vérité terrain exploitable
  - Exemple : le réseau d'une petite agence, quelques dizaines de flux par jour
- **Problème bien défini** : une règle ou un seuil suffit
  - Exemple : interdire Telnet (port 23) est une règle de pare-feu, pas un modèle
- **Explicabilité exigée** : audit, conformité, décision à fort impact
  - Exemple : couper l'accès d'un client doit pouvoir être justifié
- **Contraintes strictes** de latence ou de ressources
  - Exemple : classer chaque paquet à 100 Gb/s sur un équipement sans processeur dédié

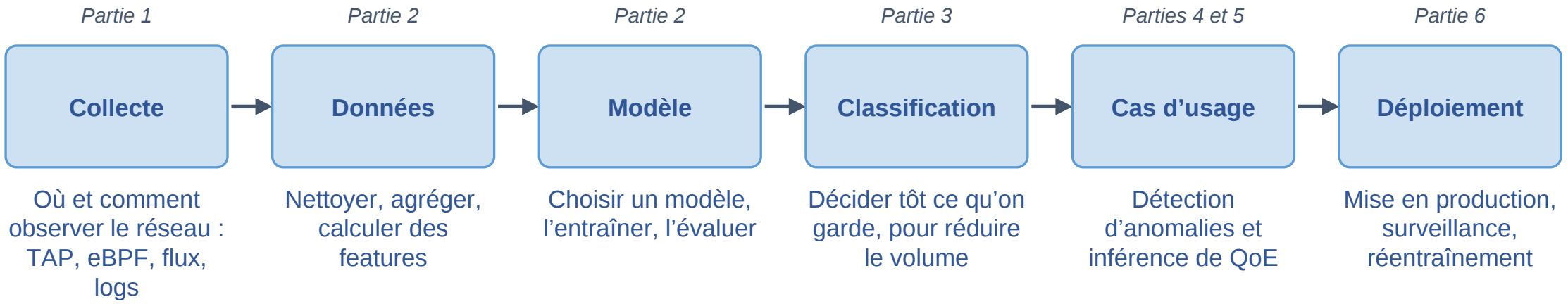
Règle d'or : toujours commencer par une baseline simple. Le ML doit la battre pour se justifier.

Ce que l’IA apporte à ces outils

Outil	Ce que l’IA apporte	Exemple
Supervision	Seuils adaptatifs, prévision de charge	Un pic à 3 h du matin est détecté, alors qu’un seuil fixe ne dit rien
IDS / IPS	Détection d’attaques inconnues par le comportement	Un scan lent, jamais vu auparavant
NDR	Profils de trafic appris automatiquement	Un poste qui parle soudain à de nouvelles destinations
SIEM	Alertes de modèles en plus des règles ; UEBA	Un utilisateur qui se connecte à des heures inhabituelles
SOAR	Aide à la décision et priorisation	Traiter d’abord les alertes à plus fort risque

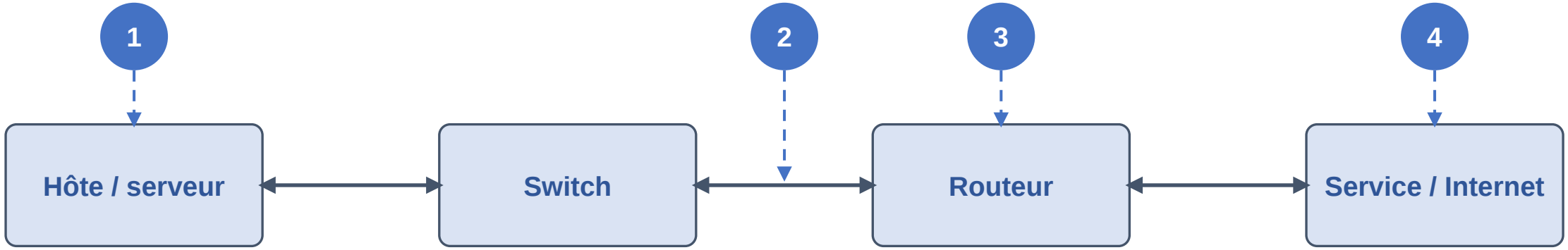
Fil rouge : le pipeline

Chaque partie du cours correspond à un maillon de cette chaîne.



1. Collecte des données

Où observer un réseau ?



1 Sur l'hôte

eBPF, logs système, conteneurs. Contexte processus et socket.

2 Sur le lien

TAP, port mirroring. Paquets complets, vue réseau.

3 Sur l'équipement

SNMP, streaming telemetry, NetFlow/IPFIX, sFlow.

4 Dans l'application

Logs, télémétrie client, SIEM. Vue de l'utilisateur.

Typologie des sources de données

Source	Granularité	Volume	Ce qu'on voit	Outils
Paquets (pcap)	Paquet	Très élevé	Contenu si non chiffré	TAP, SPAN, tcpdump
Flux	Flux (5-tuple)	Moyen	Métadonnées L3/L4	NetFlow, IPFIX, sFlow
Télémétrie	Compteurs, état	Faible	Santé des équipements	SNMP, gNMI, INT
Logs	Événement	Variable	Système et applications	syslog, SIEM
Mesures actives	Sonde	Faible	Bout-en-bout, volontaire	ping, traceroute, iperf3

Passif : on observe le trafic existant, sans l'influencer. Actif : on injecte des sondes (mesure ciblée, mais charge ajoutée).

Exemple 1 : un paquet

Un paquet = des en-têtes (qui parle à qui) + une charge utile (les données, souvent chiffrées).

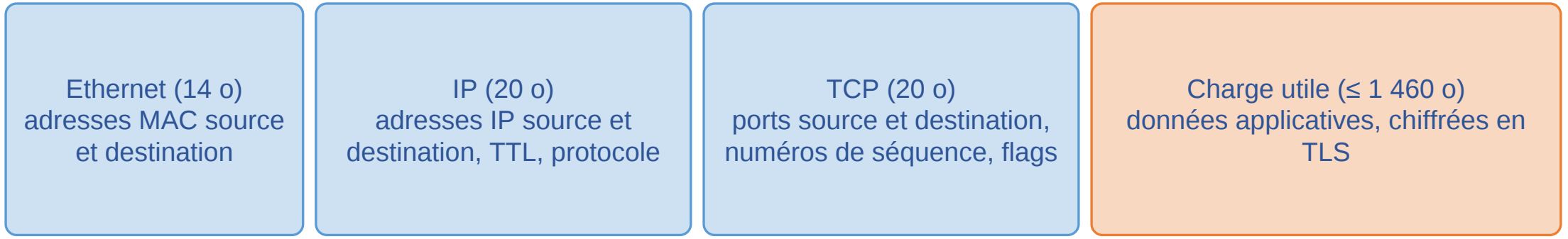


Schéma hors échelle

Ce que voit tcpdump pour un paquet :

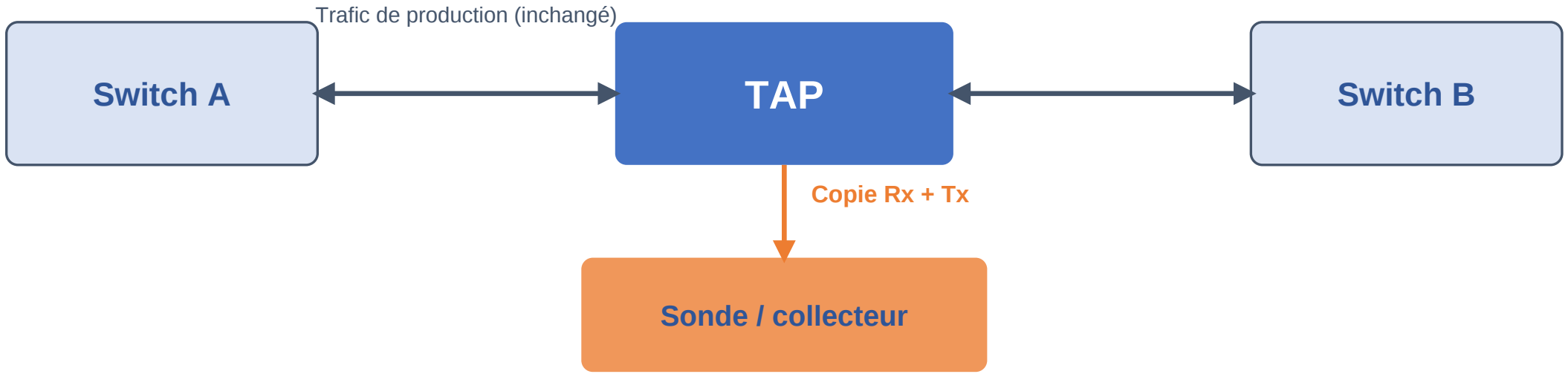
```
14:02:11.532 IP 192.0.2.10.52144 > 203.0.113.7.443: Flags [P.], seq 1:518, ack 1, win 502, length 517
```

Exemple 2 : flux, télémétrie, log, mesure active

Source	Exemple concret	Ce que ça résume
Flux	192.0.2.10:52144 → 203.0.113.7:443, TCP, 48 paquets, 61 200 octets, durée 12 s	Un enregistrement remplace 48 paquets
Télémétrie	ifInOctets (interface eth0) = 8 412 903 211, lu toutes les 60 s	Un compteur d'interface, sans détail par paquet
Log	Oct 4 14:02:11 srv1 sshd[812]: Failed password for root from 203.0.113.7	Un événement système ou applicatif
Mesure active	ping 203.0.113.7 : ttl=57, temps=18,4 ms	Une mesure de bout en bout, déclenchée volontairement

Du paquet au log, le volume diminue mais le détail aussi : c'est le compromis de la collecte.

Le TAP : copier le trafic sans le perturber



Passif

Paquets complets, horodatage précis : la fidélité est maximale.

Sans impact

Le trafic de production n'est pas modifié, même si la sonde tombe.

Limites

Coût, accès physique au lien, volume énorme à traiter ou à stocker.

TAP ou port mirroring (SPAN) ?

TAP

- Copie matérielle (optique : séparateur, cuivre : actif)
- Aucune perte, horodatage fiable
- Coût et intervention physique
- Packet brokers : agréger, filtrer, dédupliquer

Port mirroring (SPAN)

- Fonction du switch, configurable à distance
- Peut perdre des paquets (sur-souscription)
- Priorité basse, charge CPU/ASIC, horodatage altéré
- RSPAN / ERSPAN : miroir à travers le réseau

À haut débit en production : TAP + packet broker. En lab ou en campus : SPAN suffit souvent.

Capturer en pratique

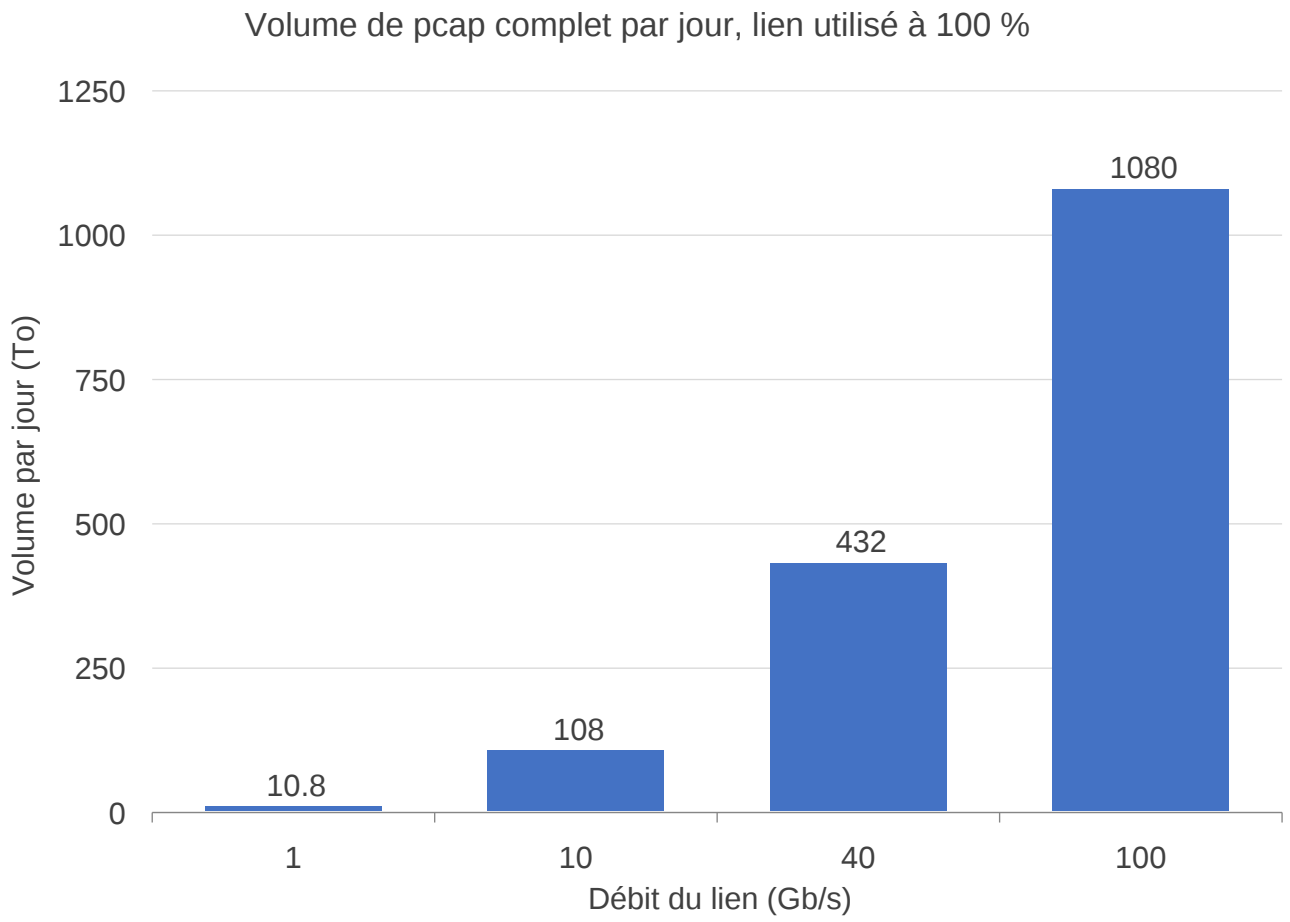
```
# Capture limitée aux en-têtes (96 octets/paquet)
tcpdump -i eth0 -s 96 -w capture.pcap 'tcp port
443'

# Rejouer et résumer en connexions (Zeek)
zeek -r capture.pcap
head conn.log

# Extraire des champs avec tshark
tshark -r capture.pcap -T fields \
  -e ip.src -e ip.dst \
  -e frame.len -e frame.time_epoch
```

- **tcpdump** : capture, filtres BPF
- **-s 96** : on garde les en-têtes, pas la charge utile
- **Zeek** : transforme le pcap en logs structurés (conn.log)
- **tshark** : extraction de champs pour le ML

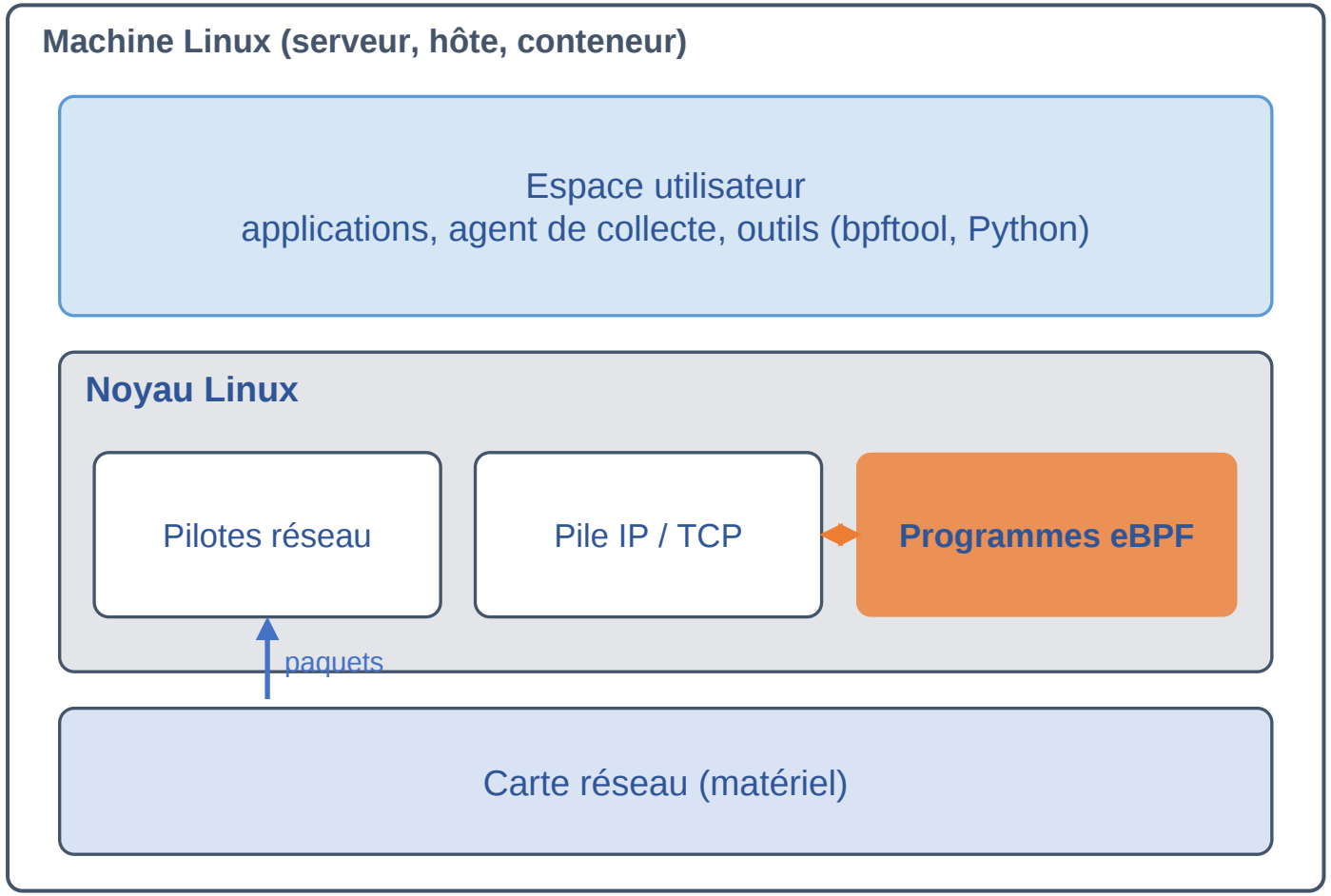
Le défi du volume : combien de données par jour ?



- **Ce que montre le graphique** : la quantité de paquets à stocker si on capture tout, selon le débit du lien
- **Calcul** : débit × 86 400 s ÷ 8. À 1 Gb/s : 0,125 Go/s, soit 10,8 To par jour
- **À 100 Gb/s** : environ 1 Po par jour
- **Lien avec la suite** : tout garder est impossible, d'où le filtrage à la source (eBPF), les flux et la classification

Source : calcul théorique (lien saturé, sans compression ni en-têtes de capture). Il s'agit d'un ordre de grandeur, pas d'une mesure.

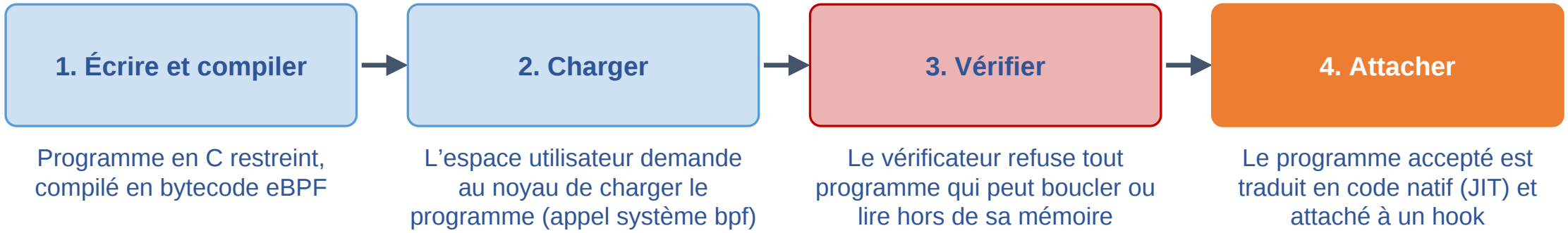
eBPF : où s’exécute-t-il ?



- **eBPF** : une machine virtuelle dans le noyau Linux
- On y charge de **petits programmes**
- Ils s’exécutent quand un **événement** survient (paquet reçu, appel système…)
- Sans modifier le noyau ni écrire de module

Pourquoi eBPF est sûr : le chemin d'un programme

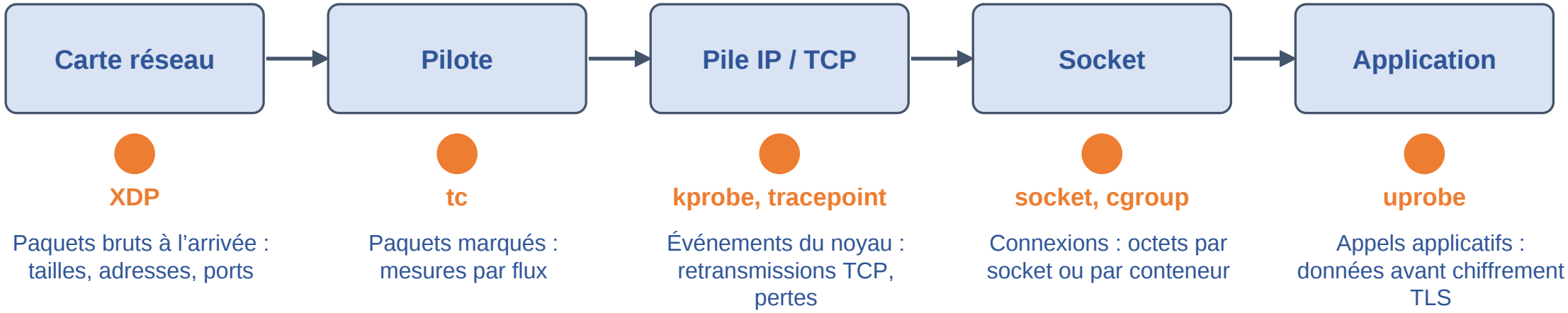
Objectif : comprendre pourquoi on peut charger du code dans le noyau sans risquer de le faire planter.



- **Résultat** : un programme eBPF ne peut pas faire planter le noyau
- **Conséquence** : on peut le charger et le retirer à chaud, sans redémarrage

Qu’est-ce qu’un hook ?

Un hook (point d’accroche) est un endroit précis du noyau où l’on peut attacher un programme eBPF. Les points orange ci-dessous sont des hooks : à chaque passage d’un paquet, le noyau exécute le programme attaché, qui peut alors observer.



Sur chaque hook, on observe un type de donnée différent.

Les points d’accroche eBPF pour le réseau

Hook	Position	Usage typique
XDP	Pilote de la carte, avant allocation du paquet	Filtrage, anti-DDoS, compteurs à haut débit
tc (ingress / egress)	Couche de contrôle de trafic	Classification, marquage, mesures par flux
Socket / cgroup	Sockets et conteneurs	Contrôle par conteneur, observabilité L4
kprobes / tracepoints	Fonctions du noyau	Retransmissions TCP, pertes, latences
uprobes	Fonctions applicatives	Contexte applicatif (ex. bibliothèque TLS)

Plus on est bas dans la pile (XDP), plus c’est rapide, mais moins on a de contexte.

Qu’est-ce qu’une map eBPF ?

- **Map** : une table de données (clé → valeur) gardée dans le noyau
- Un programme eBPF s’exécute sans mémoire d’une fois sur l’autre : la map conserve l’état
- Le noyau et l’espace utilisateur peuvent la lire et l’écrire
- Types usuels : table de hachage, tableau

Exemple : compter les paquets par IP source

Clé : IP source	Valeur : paquets
192.0.2.10	4 812
198.51.100.7	37
203.0.113.9	1 205

À chaque paquet, le programme met à jour la ligne correspondante.

Programme eBPF et map eBPF : deux rôles

	Programme eBPF	Map eBPF
Nature	Code (bytecode vérifié)	Structure de données clé → valeur
Rôle	Traite chaque événement (paquet, appel système)	Garde l'état : compteurs, tables, résultats
Durée de vie	Une exécution très courte, sans mémoire propre	Persiste tant que la map existe
Accès	S'exécute dans le noyau	Lue et écrite par le noyau et l'espace utilisateur
Exemple	« Incrémenter le compteur de cette IP »	Table IP source → nombre de paquets

Le programme calcule, la map se souvient.

Agréger à la source avec les maps eBPF

1. **Paquets** : arrivent sur l'interface
2. **Programme eBPF** : extrait le 5-tuple, met à jour la map
3. **Map eBPF** : clé : 5-tuple. Valeur : paquets, octets, horodatage
4. **Lecture** : L'espace utilisateur lit la map toutes les N secondes
5. **Features** : tailles, débits, durées par flux

On remonte quelques kilo-octets de compteurs au lieu de gigaoctets de paquets.

Exemple : compter les paquets par IP source (XDP)

```
// includes omis
struct { __uint(type, BPF_MAP_TYPE_HASH);
  __type(key, __u32); __type(value, __u64);
  __uint(max_entries, 65536); } pkts SEC(".maps");


SEC("xdp")
int count(struct xdp_md *ctx) {
  void *data = (void *)(long)ctx->data;
  void *end  = (void *)(long)ctx->data_end;
  struct ethhdr *eth = data;
  if ((void *)(eth + 1) > end) return XDP_PASS;
  if (eth->h_proto != bpf_htons(ETH_P_IP))
    return XDP_PASS;
  struct iphdr *ip = (void *)(eth + 1);
  if ((void *)(ip + 1) > end) return XDP_PASS;
  __u64 one = 1, *v = bpf_map_lookup_elem(&pkts, &ip->saddr);
  if (v) __sync_fetch_and_add(v, 1);
  else bpf_map_update_elem(&pkts, &ip->saddr, &one, BPF_ANY);
  return XDP_PASS;
}
```

- La map est une table de hachage : IP source → compteur
- Chaque accès mémoire est vérifié (bornes)
- **XDP_PASS** : le paquet continue normalement
- Compilation : clang -O2 -target bpf

Ce que le TAP ne voit pas, eBPF le voit

TAP / SPAN

- Voit : tous les paquets du lien
- Ne voit pas : quel processus ou conteneur
- Ne voit pas : le trafic interne à l'hôte
- Charge utile chiffrée = opaque

eBPF

- Voit : processus, conteneur, socket, appel système
- Peut observer avant le chiffrement (uprobes sur une bibliothèque TLS)
- Ne voit que les hôtes équipés
- Sensible : données personnelles, droits élevés requis

Quelques outils fondés sur eBPF

Outil	Rôle	Ce qu'il observe
Cilium / Hubble	Observabilité réseau dans Kubernetes	Les flux entre pods et services
Pixie	Observabilité des applications Kubernetes	Requêtes HTTP, DNS, sans modifier le code
Falco, Tetragon	Sécurité à l'exécution	Appels système suspects
bpftrace	Diagnostic ponctuel	N'importe quel événement du noyau

Ce sont des applications d'eBPF, pas des alternatives au TAP : ils observent l'intérieur de la machine.

Comparatif : TAP / SPAN et eBPF

Critère	TAP / SPAN	eBPF
Position	Sur le lien	Sur l'hôte
Visibilité	Tout le trafic du lien	Uniquement les hôtes équipés
Contenu	Paquets complets	Au choix : agrégats ou événements
Contexte	Aucun (réseau seul)	Processus, conteneur, socket
Coût	Matériel et intervention	Logiciel, déploiement sur le parc
Limite	Débit, stockage	Droits élevés, compatibilité noyau

Des paquets aux flux : agréger pour réduire

- Jusqu'ici (TAP, eBPF) : on capture des **paquets** ou des événements sur l'hôte
- Problème : le volume est trop grand (voir le graphique précédent)
- Idée : regrouper les paquets d'une même communication et ne garder qu'un résumé
- **Flux** : ensemble de paquets qui partagent le même 5-tuple (IP source, IP destination, port source, port destination, protocole)

Paquet	IP source	IP dest.	Port src	Port dst	Proto	Taille
1	192.0.2.10	203.0.113.7	52144	443	TCP	74 o
2	192.0.2.10	203.0.113.7	52144	443	TCP	66 o
3	192.0.2.10	203.0.113.7	52144	443	TCP	517 o
4	192.0.2.10	203.0.113.7	52144	443	TCP	1 460 o



Un flux
192.0.2.10:52144 → 203.0.113.7:443
TCP, 4 paquets, 2 117 octets

Les outils suivants (NetFlow, IPFIX, sFlow) construisent ces flux directement sur le routeur, sans capturer les paquets.

Qu’est-ce que NetFlow et IPFIX ?

- **NetFlow** : technologie de Cisco (versions 5 et 9) où le routeur construit lui-même des enregistrements de flux
- **IPFIX** : standard de l’IETF (RFC 7011), issu de NetFlow v9 et indépendant du constructeur
- **Principe** : l’équipement garde en mémoire les flux en cours, puis exporte chaque enregistrement terminé vers un collecteur
- **Différence avec sFlow** : NetFlow et IPFIX comptent en principe tous les paquets (l’échantillonnage est optionnel) ; sFlow échantillonne toujours

NetFlow et IPFIX : à quoi ça sert



À quoi ça sert

- Voir qui parle à qui, et combien
- Détecter scans et DDoS
- Planifier la capacité, enquêter après un incident

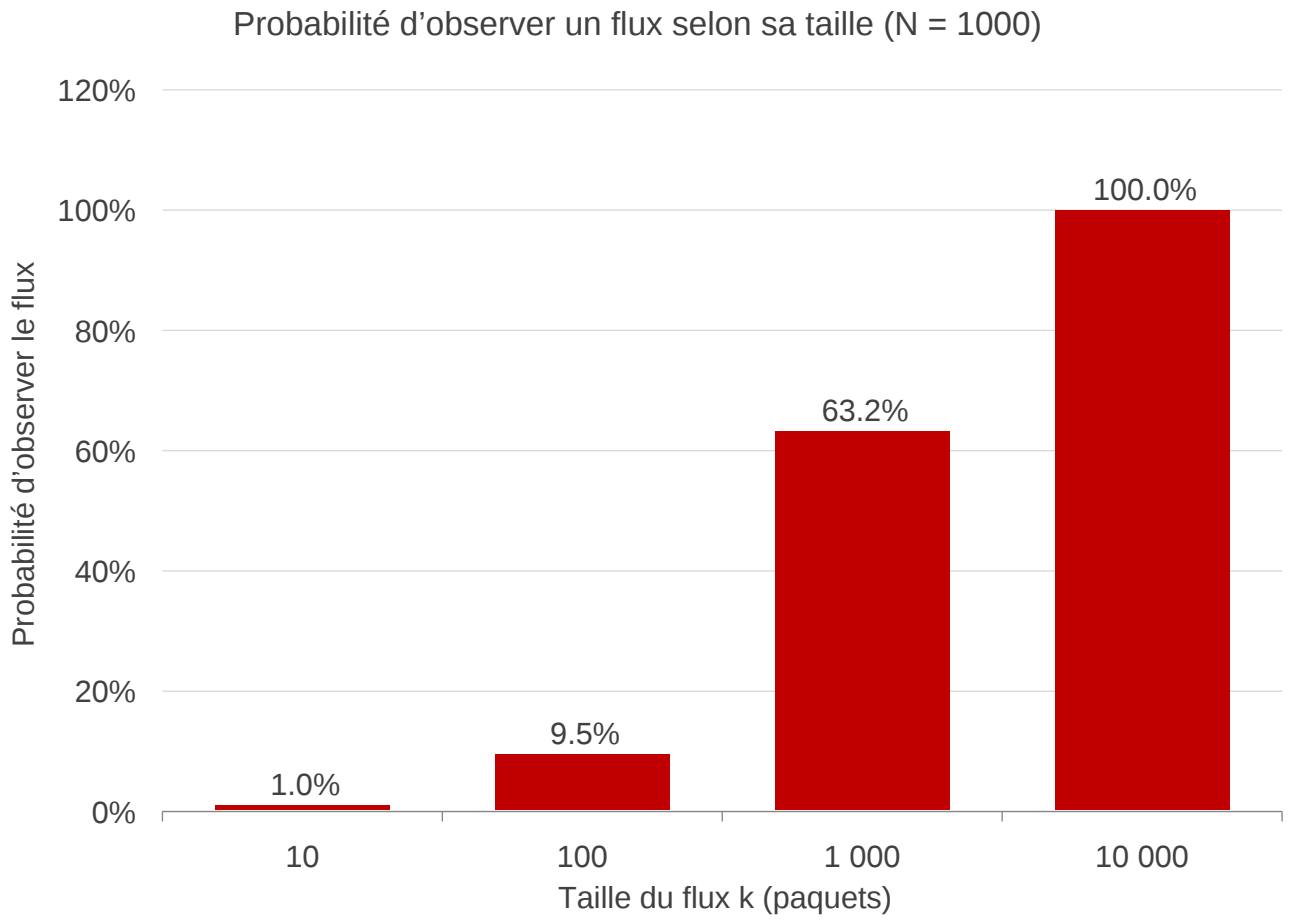
Réglages

- Timeouts actif et inactif
- Taux d’échantillonnage (optionnel)
- Champs exportés (IPFIX, extensible)

sFlow : échantillonner plutôt que tout compter

- **sFlow** : technologie d'échantillonnage présente sur de nombreux commutateurs et routeurs
- **Principe** : l'équipement copie **1 paquet sur N** (ex. N = 1000) et l'envoie à un collecteur, avec les compteurs d'interface
- **Avantage** : charge très faible, même à 100 Gb/s
- **Limite** : la vue est statistique, on estime le trafic en multipliant par N
- **Question** : quelle part des petits flux échappe à cette observation ?

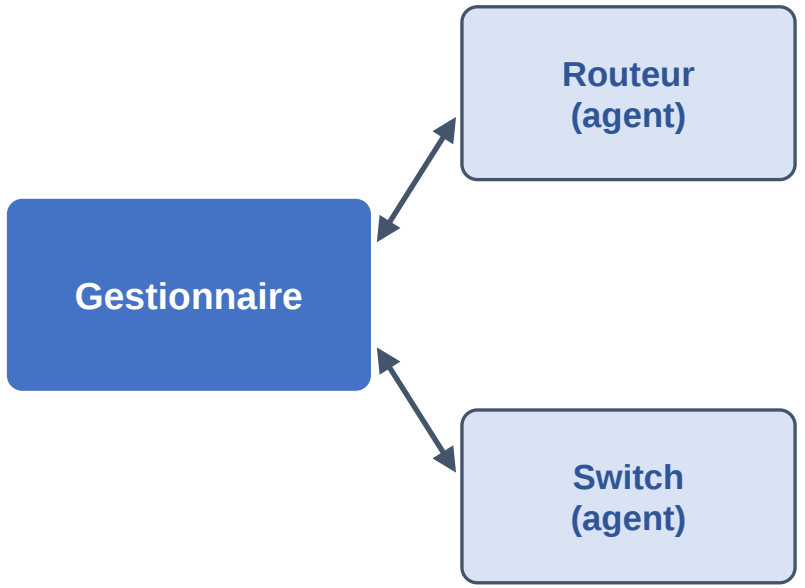
Que perd-on avec l’échantillonnage ?



- **Question** : avec 1 paquet sur 1000, voit-on un flux de k paquets ?
- **Formule** : P(voir au moins 1 paquet) = 1 − (1 − 1/N)^k
- **Exemple** : k = 100 donne P = 1 − 0,999^100 ≈ 9,5 %
- **Lecture** : un gros flux (10 000 paquets : DDoS, vidéo) est vu à coup sûr ; un scan de 3 paquets par cible n’a que 0,3 % de chances d’être vu

Qu’est-ce que SNMP ?

- **SNMP** (Simple Network Management Protocol) : protocole standard de supervision des équipements, créé en 1988
- **Gestionnaire** : le serveur de supervision, qui interroge
- **Agent** : un logiciel sur chaque équipement, qui répond
- **MIB** : le catalogue des grandeurs mesurables (octets par interface, CPU, état des ports)



GET : lire un compteur
TRAP : alerte envoyée par l’équipement

SNMP : un exemple de requête

```
# Lire le compteur d'octets reçus de l'interface 2
snmpget -v2c -c public 192.0.2.1 \
  IF-MIB::ifInOctets.2

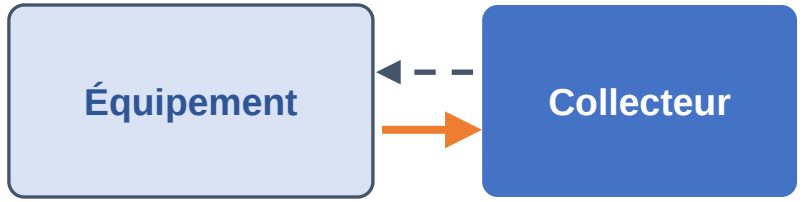
# Réponse de l'agent
IF-MIB::ifInOctets.2 = Counter32: 8412903211

# OID correspondant
1.3.6.1.2.1.2.2.1.10.2
```

- **Opérations** : GET, GETBULK, SET, TRAP
- **OID** : l'identifiant de la grandeur (ici ifInOctets)
- **Transport** : UDP, ports 161 et 162
- **Versions** : v2c (communauté en clair), v3 (authentifiée et chiffrée)
- **Mode** : polling, le gestionnaire interroge à intervalle régulier

Qu’est-ce que gNMI ?

- **gNMI** (gRPC Network Management Interface) : protocole moderne de supervision et de configuration, défini par OpenConfig
- **Principe** : au lieu d’être interrogé, l’équipement **publie** ses mesures en continu (streaming telemetry)
- **Données** : décrites par des modèles YANG et adressées par un chemin (ex. /interfaces/interface/state/counters)
- **Pourquoi après SNMP** : mesures plus fréquentes (secondes) et moins de charge sur l’équipement



Subscribe (abonnement), puis flux continu de mesures

gNMI : un exemple d’abonnement

```
# Souscrire aux compteurs d’interface,
# une mesure toutes les 10 secondes
gnmic -a 192.0.2.1:57400 subscribe \
  --path "/interfaces/interface/state/counters" \
  --mode stream \
  --stream-mode sample \
  --sample-interval 10s
```

- **Subscribe** : l’équipement publie en continu, à intervalle fixe ou sur changement
- **Transport** : gRPC (HTTP/2) avec TLS, encodage protobuf
- **Opérations** : Get, Set, Capabilities, Subscribe
- **Intérêt** : fréquence fine et moins de charge que le polling SNMP

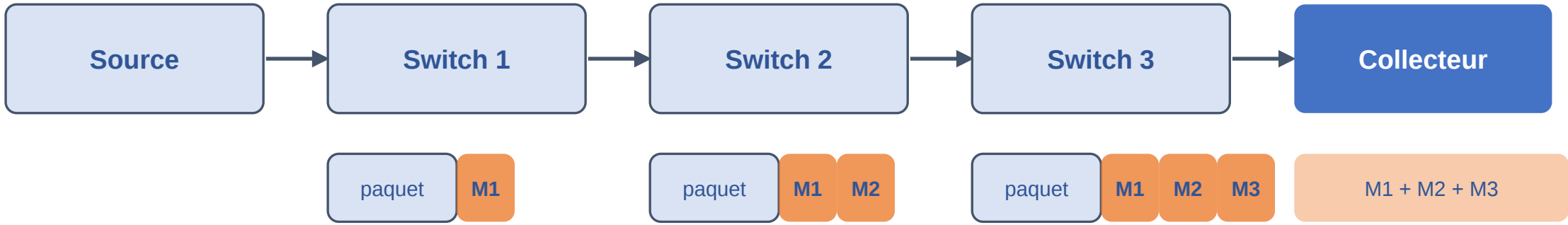
SNMP ou streaming telemetry ?

	SNMP	Streaming telemetry (gNMI)
Modèle	Pull : le collecteur interroge	Push : l'équipement publie
Fréquence	Minutes (polling)	Secondes ou moins, sur événement
Modèle de données	MIB	YANG / OpenConfig
Encodage	ASN.1 / BER	Protobuf sur gRPC
Charge équipement	Élevée si polling fréquent	Plus efficace à grande échelle

La télémétrie en continu fournit des séries temporelles fines, directement exploitables par des modèles.

INT : la télémétrie dans les paquets

INT (In-band Network Telemetry) : chaque équipement traversé ajoute ses propres mesures dans le paquet lui-même.



M = métadonnées d'un équipement : identifiant, horodatages, longueur de file

- **Atout** : vue précise du chemin et des délais de file, paquet par paquet
- **Limites** : matériel programmable (P4), octets ajoutés à chaque paquet

Mesures actives et mesures passives

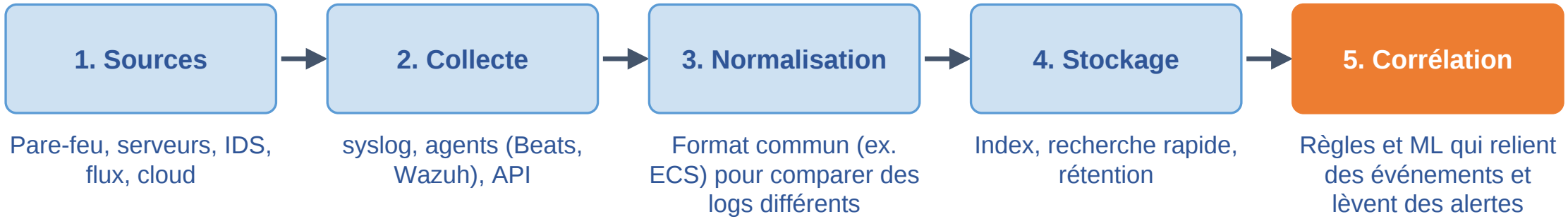
- **Mesures actives** : on injecte des sondes (ping, traceroute, iperf3, RIPE Atlas) pour mesurer le bout en bout
  - Contrôlées et reproductibles, mais elles ajoutent de la charge et ne reflètent pas toujours le trafic réel
- **Mesures passives** : on observe le trafic existant (TAP, flux, télémétrie)
  - Non intrusives et représentatives, mais dépendantes du trafic présent
- **En pratique** : on combine les deux

Qu’est-ce qu’un SIEM ?

- **SIEM** : Security Information and Event Management. Il combine la gestion des logs (SIM) et l’analyse des événements en temps réel (SEM)
- **Il collecte** les logs de sources très différentes : pare-feu, serveurs, IDS, cloud
- **Il normalise** ces logs dans un format commun
- **Il stocke** les logs pour l’investigation et la conformité
- **Il corrèle et détecte** : règles, puis ML (UEBA)
- **Il alerte** : une console unique pour les analystes du SOC

La gestion des logs : le pipeline

Le SIEM transforme des logs dispersés en alertes exploitables, en cinq étapes.



Les étapes 3, 4 et 5 sont détaillées dans les slides suivantes ; la corrélation est le sujet de la partie 4.

Normaliser un événement

```
# Log brut (syslog)
Oct  4 14:02:11 srv1 sshd[812]: Failed password
  for root from 203.0.113.7 port 52144 ssh2

# Événement normalisé (schéma de type ECS)
{
  "@timestamp": "2026-10-04T14:02:11Z",
  "host.name": "srv1",
  "event.category": "authentication",
  "event.outcome": "failure",
  "user.name": "root",
  "source.ip": "203.0.113.7",
  "source.port": 52144
}
```

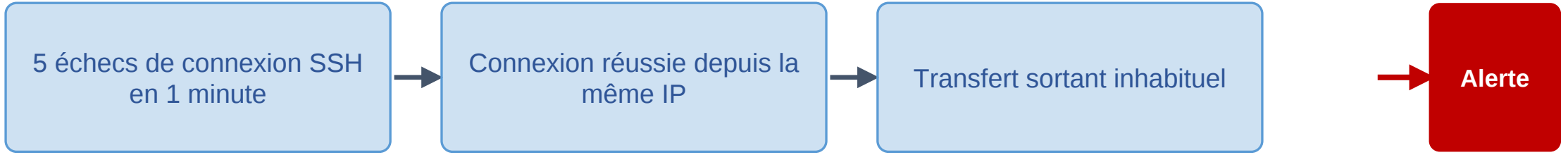
- Champs communs : corrélation entre sources hétérogènes
- **ECS** (Elastic Common Schema) : noms de champs standardisés
- Parsing : grok, décodeurs Wazuh, pipelines d'ingestion
- La qualité du parsing fixe la qualité des détections

Stockage et conservation des logs

- **Pourquoi stocker** : enquêter après un incident, prouver la conformité, entraîner des modèles
- **Indexer** : rendre la recherche rapide sur des millions d'événements
- **Niveaux de stockage** : récent (rapide, cher), ancien (lent, peu cher), archive
- **Rétention** : durée fixée par la loi et la politique de sécurité
- **Coût** : l'ingestion est souvent facturée au volume, d'où l'intérêt de filtrer et de résumer avant d'envoyer (partie 3)
- **Lien avec l'IA** : ces logs historiques servent de données d'entraînement (partie 2)

La corrélation : relier des événements

Un événement isolé est banal ; c'est la séquence qui est suspecte.

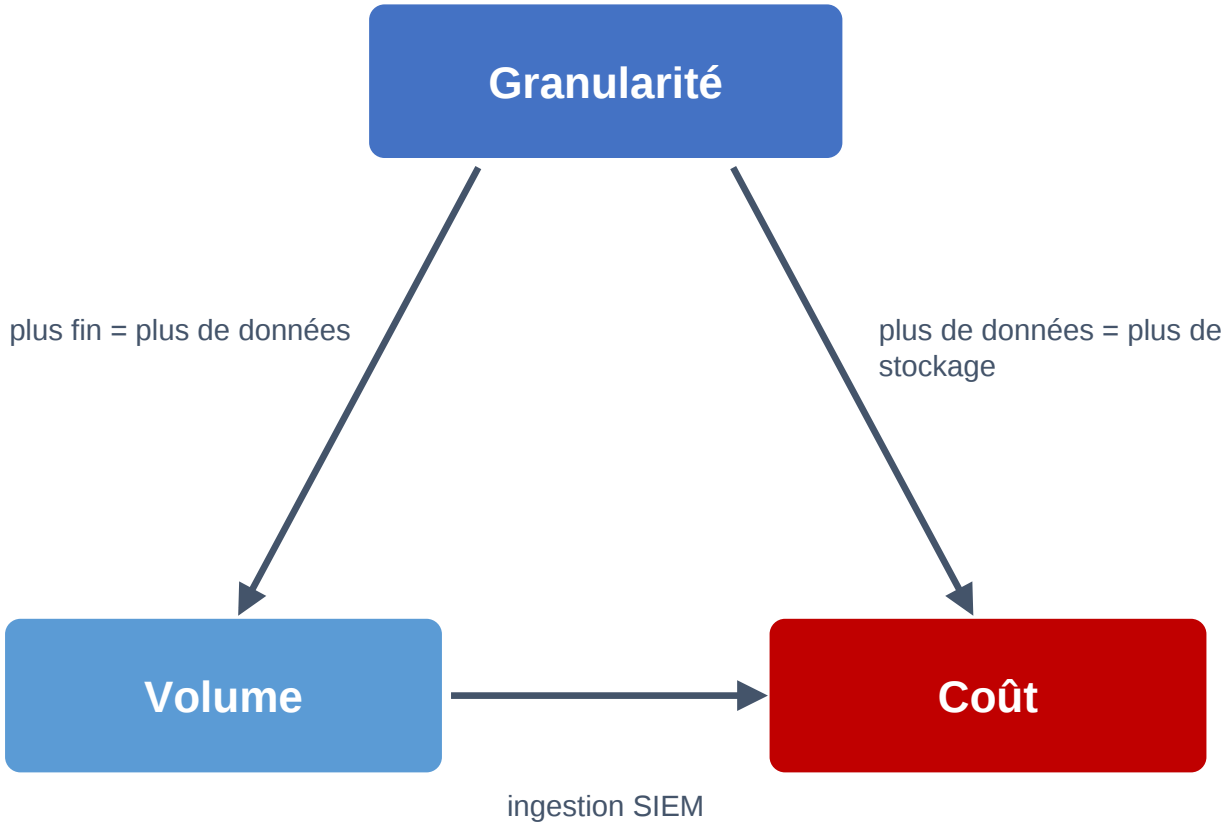


- **Règle de corrélation** : écrite à la main par un expert (ex. cette séquence en moins de 10 minutes)
- **Corrélation apprise** : un modèle repère des séquences inhabituelles sans règle écrite
- Les deux approches sont comparées dans la partie 4

Panorama des SIEM

Outil	Modèle	Requêtes	Particularité
Splunk	Commercial	SPL	Très répandu ; coût lié au volume ingéré
Elastic Security	Open core	KQL, EQL	Schéma ECS, modules de ML intégrés
Microsoft Sentinel	Cloud (Azure)	KQL	Facturation à l'ingestion, intégration Microsoft
IBM QRadar	Commercial	AQL	Corrélation orientée « offenses »
Wazuh	Open source	OpenSearch	HIDS + SIEM, idéal pour les TP

Granularité, volume, coût : le compromis



- **Paquets complets** : fidélité maximale, stockage ingérable
- **Flux** : bon compromis, perte de détail
- **Échantillonnage** : volume faible, biais
- **SIEM** : le coût d'ingestion pousse à filtrer ou résumer avant d'envoyer

Chiffrement : ce qui reste visible

Élément	TLS 1.2	TLS 1.3 / QUIC	Avec ECH
En-têtes IP, ports	Visibles	Visibles (QUIC : UDP/443)	Visibles
SNI (nom du site)	En clair	En clair	Chiffré
Certificat serveur	En clair	Chiffré	Chiffré
Charge utile	Chiffrée	Chiffrée	Chiffrée
Tailles, rythme, direction	Visibles	Visibles	Visibles

Le DPI classique recule ; restent les métadonnées : d'où le ML sur les tailles et les temps.

Vie privée et RGPD

- **Donnée personnelle** : une adresse IP identifie le plus souvent une personne (jurisprudence européenne).
- **Minimisation** : ne collecter que ce qui sert la finalité : en-têtes plutôt que contenu.
- **Anonymisation** : crypto-PAn : les IP sont remplacées en préservant les préfixes, donc la structure du réseau.
- **Conservation** : durée limitée, accès contrôlé, finalité documentée (sécurité, supervision).

Slide pour information : on ne détaille pas le cadre juridique dans ce cours.

Étiquetage : le vrai goulot

- **Étiqueter** : associer à chaque donnée la réponse attendue (sa classe). Exemple : ce flux est de la vidéo, ce paquet fait partie d'une attaque
- **Pourquoi c'est un goulot** : un modèle supervisé a besoin de milliers d'exemples étiquetés, rarement disponibles
- **D'où viennent les étiquettes ?**
  - Testbed, simulation : contrôlé et reproductible, mais peu réaliste
  - Règles ou IDS : rapide, mais circulaire (le modèle apprend la règle)
  - Annotation experte : fiable, mais lente et coûteuse
  - Télémétrie client : excellente vérité terrain pour la QoE (côté plateforme)

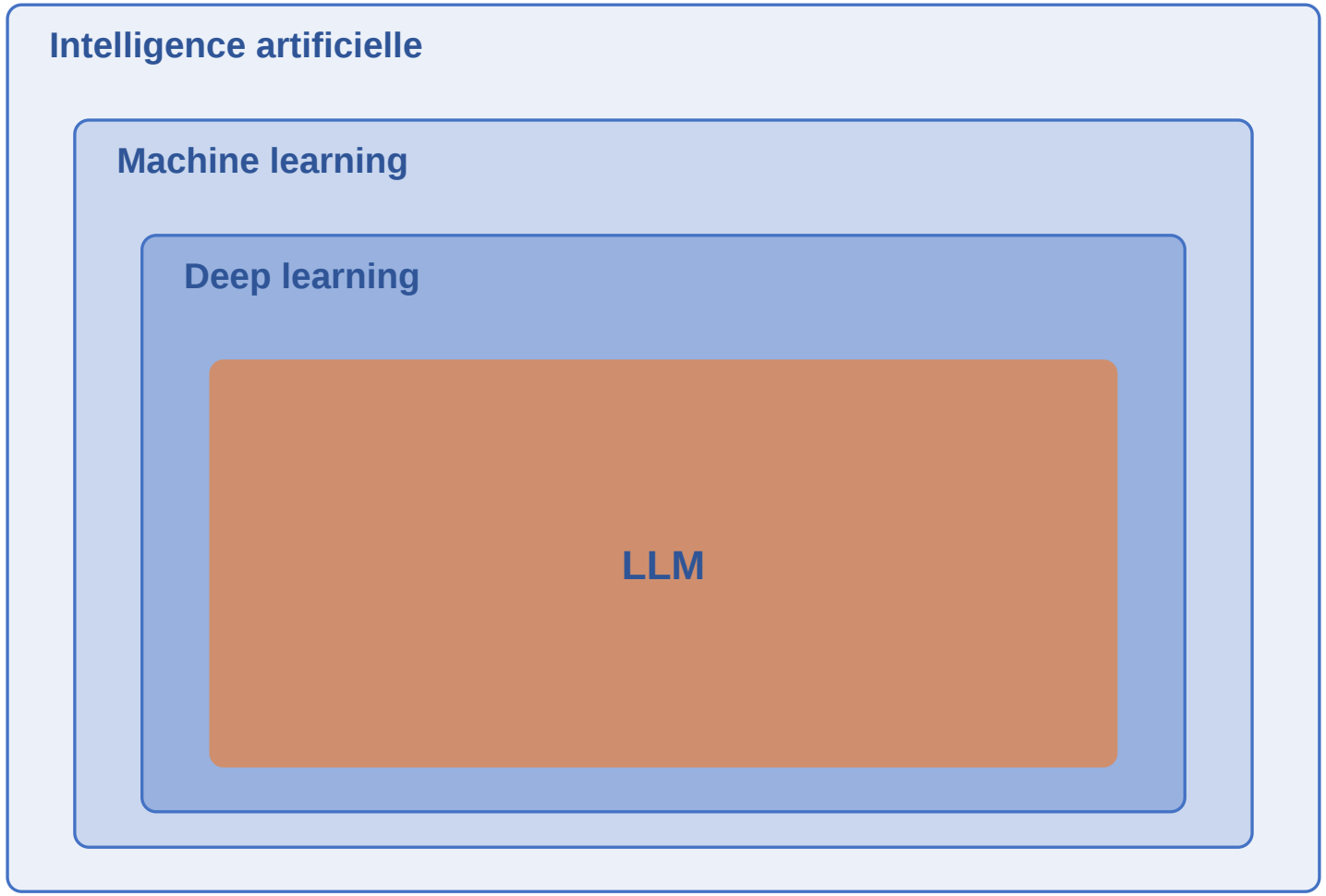
Sans vérité terrain fiable, on ne sait ni entraîner ni évaluer correctement.

À retenir : la collecte

- **Où ?** : hôte, lien, équipement, application : chaque point de vue a ses angles morts.
- **Comment ?** : TAP et SPAN pour les paquets, eBPF pour le contexte hôte, flux et télémétrie pour un volume maîtrisé.
- **À quel prix ?** : volume, chiffrement, vie privée et étiquetage dictent la suite du pipeline.

2. L’IA : entrées, sorties, modèles

Plusieurs familles : IA, ML, deep learning



- **Idée** : chaque famille est incluse dans la précédente
- **IA** : règles expertes, recherche, planification
- **ML** : apprendre à partir de données (arbres, SVM, k-means)
- **Deep learning** : réseaux profonds (CNN, LSTM, Transformers)
- **LLM** : modèles de langage (logs, assistants)

Entraînement : apprendre un modèle

Objectif : construire un modèle à partir de données passées dont on connaît la réponse.



- Se fait hors ligne, avec des données historiques
- Se refait périodiquement pour suivre l'évolution du trafic

Inférence : utiliser le modèle

Objectif : à partir de données observées sur le réseau, obtenir une prédiction.



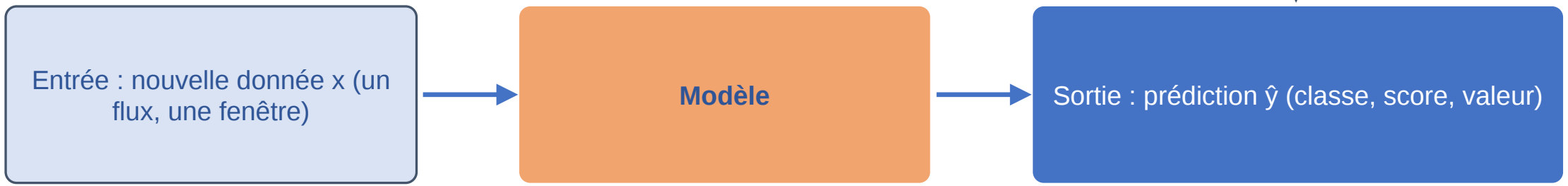
- On confronte les observations à ce que le modèle a appris (le comportement de référence)
- La prédiction en découle : « ce flux est de la vidéo », « ce flux est anormal »
- Se fait en continu : latence et débit comptent

Entraînement et inférence : vue d'ensemble

Entraînement



Inférence



En réseau : x = features d'un flux ou d'une fenêtre, ŷ = classe, score ou valeur.

Entrées : ce qu’on donne au modèle

Type	Exemple	Source (partie 1)
Paquets bruts	En-têtes, premiers octets	TAP, SPAN
Features par flux	Tailles, inter-arrivées, durée, flags TCP	NetFlow, IPFIX, eBPF
Séries temporelles	Débit par fenêtre, compteurs	SNMP, gNMI
Logs et événements	Échecs de connexion, alertes IDS	syslog, SIEM
Graphes	Qui communique avec qui	Flux, logs DNS

La source de collecte impose la granularité, donc la famille de modèles possibles.

Entrée : paquets bruts, apprendre sur les octets

- **Entrée** : en-têtes et premiers octets (ex. 784 octets ↔ image 28×28)
- **Modèles** : CNN 1D ou 2D, autoencodeurs, Transformers
- **Atout** : pas de feature engineering manuel
- **Limites** : coût de calcul, charge utile chiffrée donc peu informative
- **Attention** : un modèle qui lit les adresses IP apprend le jeu de données, pas le comportement : à masquer ou anonymiser.

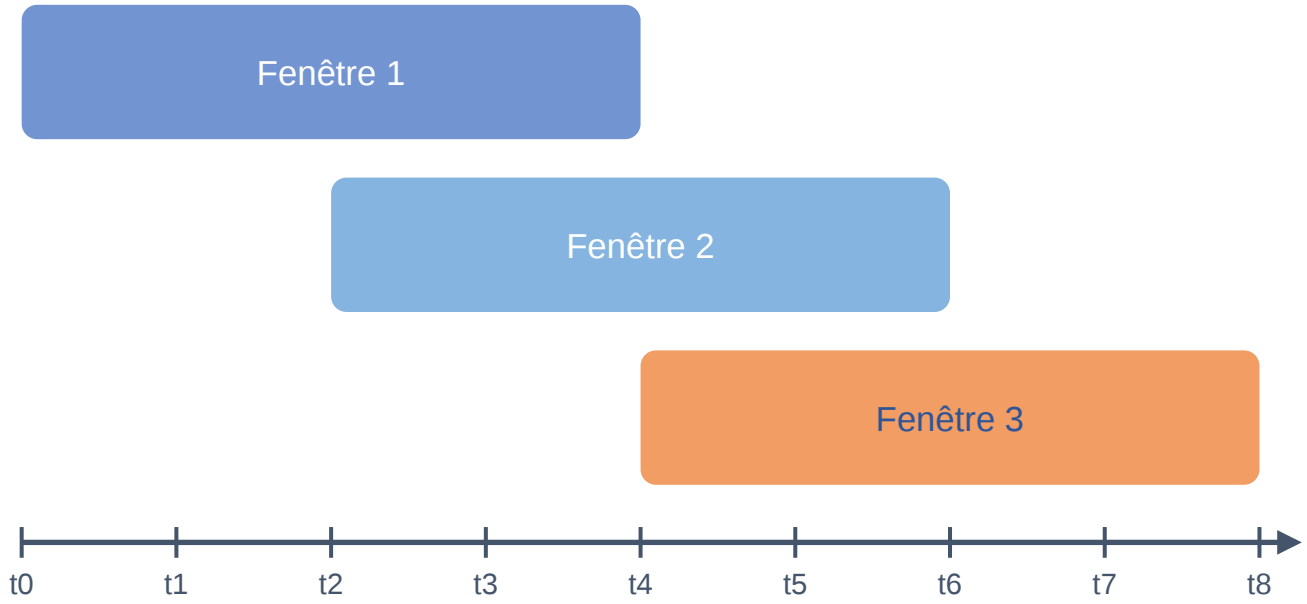
Entrée : features par flux, un exemple

Flux	Paquets	Octets	Durée (s)	Taille moy. (o)	IAT moy. (ms)	Classe
A	12	5 400	0,8	450	70	web
B	4 800	6 100 000	62	1 270	13	vidéo
C	3	180	0,01	60	5	scan

Exemple illustratif. Features usuelles : tailles (moy., σ, min, max), inter-arrivées (IAT), durée, débit, flags TCP, ratio montant/descendant. Outils : CICFlowMeter, nfstream, Zeek.

Entrée : séries temporelles et fenêtrage

Une série temporelle est une suite de mesures à intervalles réguliers (ici le débit chaque minute). Le modèle ne reçoit pas la série entière : on la découpe en fenêtres, chacune résumée en un vecteur de features.



Fenêtres glissantes (taille 4, pas 2)

Exemple : débit (Mb/s) par minute

Série : 12, 14, 13, 55, 12, 13, 14, 12

- Fenêtre 1 : 12, 14, 13, 55 → moyenne 23,5 ; max 55
- Fenêtre 2 : 13, 55, 12, 13 → moyenne 23,3 ; max 55
- Fenêtre 3 : 12, 13, 14, 12 → moyenne 12,8 ; max 14

Chaque ligne est un vecteur de features donné au modèle.

Sorties : ce qu’on attend du modèle

Sortie	Exemple	Tâche
Une classe	Application (vidéo, web, VoIP) ; bénin ou malveillant	Classification
Un score	Degré d’anomalie d’un flux ou d’un hôte	Détection d’anomalies
Une valeur continue	Résolution vidéo, délai de démarrage, débit	Régression
Une série	Charge prévue dans l’heure	Prévision
Une alerte	Score, explication et contexte	Aide à la décision

Les familles de tâches (fil conducteur)

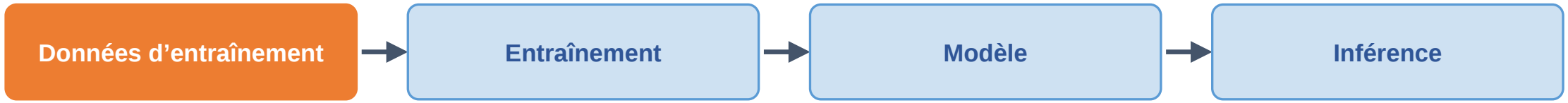
Tâche	Entrée typique	Sortie	Modèle courant	Partie
Classification de flux	Features des premiers paquets	Classe d'application	Forêt aléatoire, gradient boosting	3
Détection d'anomalies	Features par fenêtre, logs	Score ou alerte	Isolation Forest, autoencodeur	4
Inférence de QoE	Tailles et rythme des segments	Résolution, interruptions	Gradient boosting, LSTM	5

Ce tableau sert de fil conducteur : on y revient dans chaque partie.

Supervisé, non supervisé, semi-supervisé

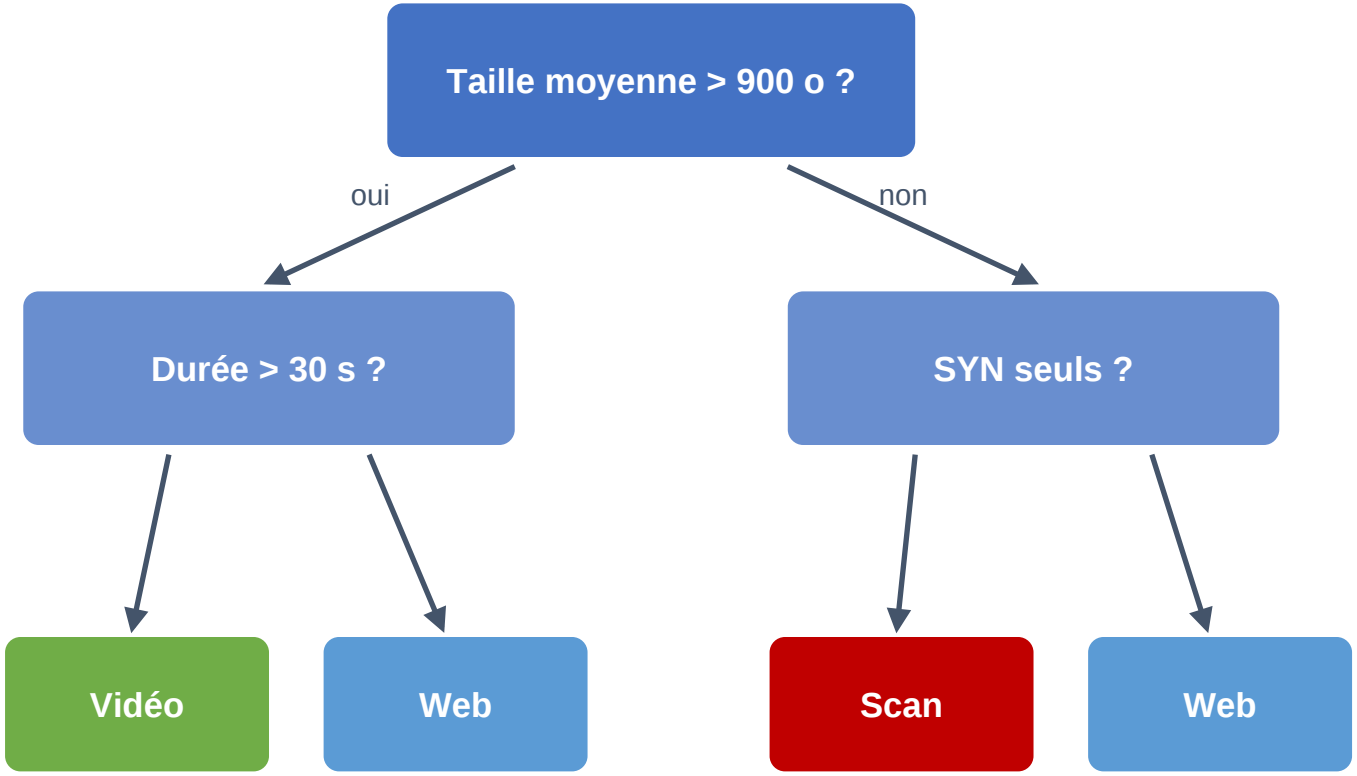
- **Supervisé** : données étiquetées. Précis sur les cas connus, mais exige des labels. Ex. : classer les applications
- **Non supervisé** : aucun label. Modélise le « normal » ou regroupe. Beaucoup de faux positifs. Ex. : détecter des anomalies
- **Semi-supervisé** : apprend sur du trafic normal seulement, ou avec peu de labels. Compromis fréquent en sécurité

Ce qui change : les données d'entraînement. Le reste du pipeline est identique.



Supervisé : données + étiquettes
Non supervisé : données seules
Semi-supervisé : peu d'étiquettes, ou trafic normal seulement

Arbres, forêts et gradient boosting



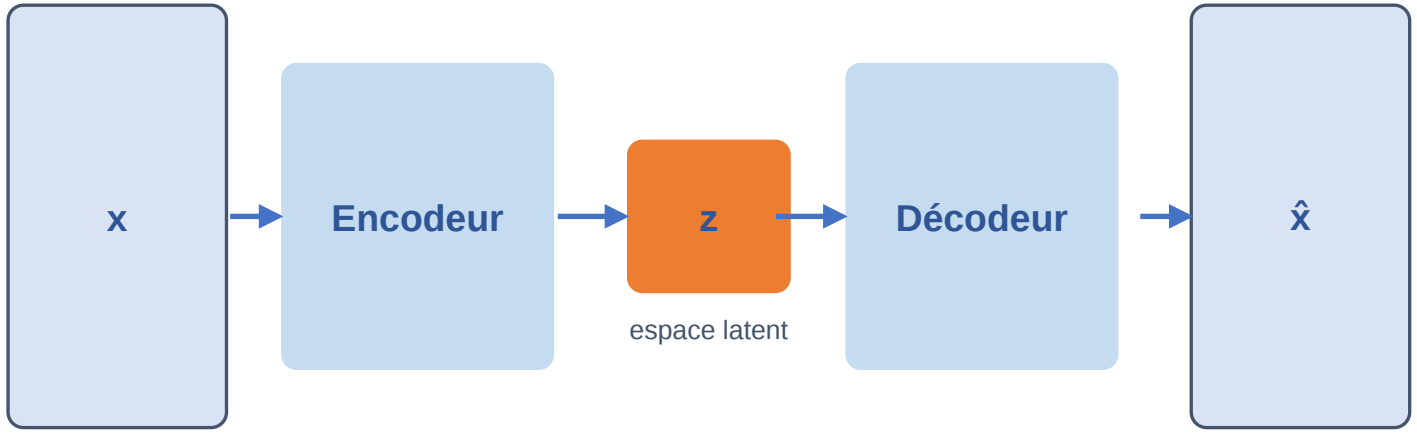
Un arbre de décision pour classer des flux (exemple simplifié)

- **Arbre de décision** : une suite de questions sur les features ; chaque feuille donne une classe
- **Cas d'usage** : classer un flux (vidéo, web, scan) à partir de la taille moyenne de ses paquets, de sa durée, de ses flags
- **Forêt aléatoire** : plusieurs arbres qui votent, plus robuste
- **Gradient boosting** (XGBoost, LightGBM) : arbres construits l'un après l'autre pour corriger les erreurs des précédents
- **Limite** : pas de notion de séquence

Autres modèles classiques

Modèle	Principe	Usage réseau	Limite
Régression logistique	Frontière linéaire, probabilités	Baseline rapide, interprétable	Relations non linéaires
SVM	Marge maximale (noyaux)	Classification de flux	Passage à l'échelle
k-NN	Voisins les plus proches	Prototype, peu de données	Coût à l'inférence
k-means, DBSCAN	Regroupement sans label	Profils de trafic, outliers	Choix de k, de la distance

Détection non supervisée : modéliser le normal



- **x** : les features d'un flux (ex. taille moyenne 450 o, durée 0,8 s, 12 paquets)
- **x̂** : ce que l'autoencodeur reconstruit après avoir compressé x
- Flux normal : x̂ ≈ x, erreur faible
- Flux anormal : x̂ s'écarte de x, erreur élevée → alerte

Score d'anomalie = erreur de reconstruction $\|x - \hat{x}\|^2$

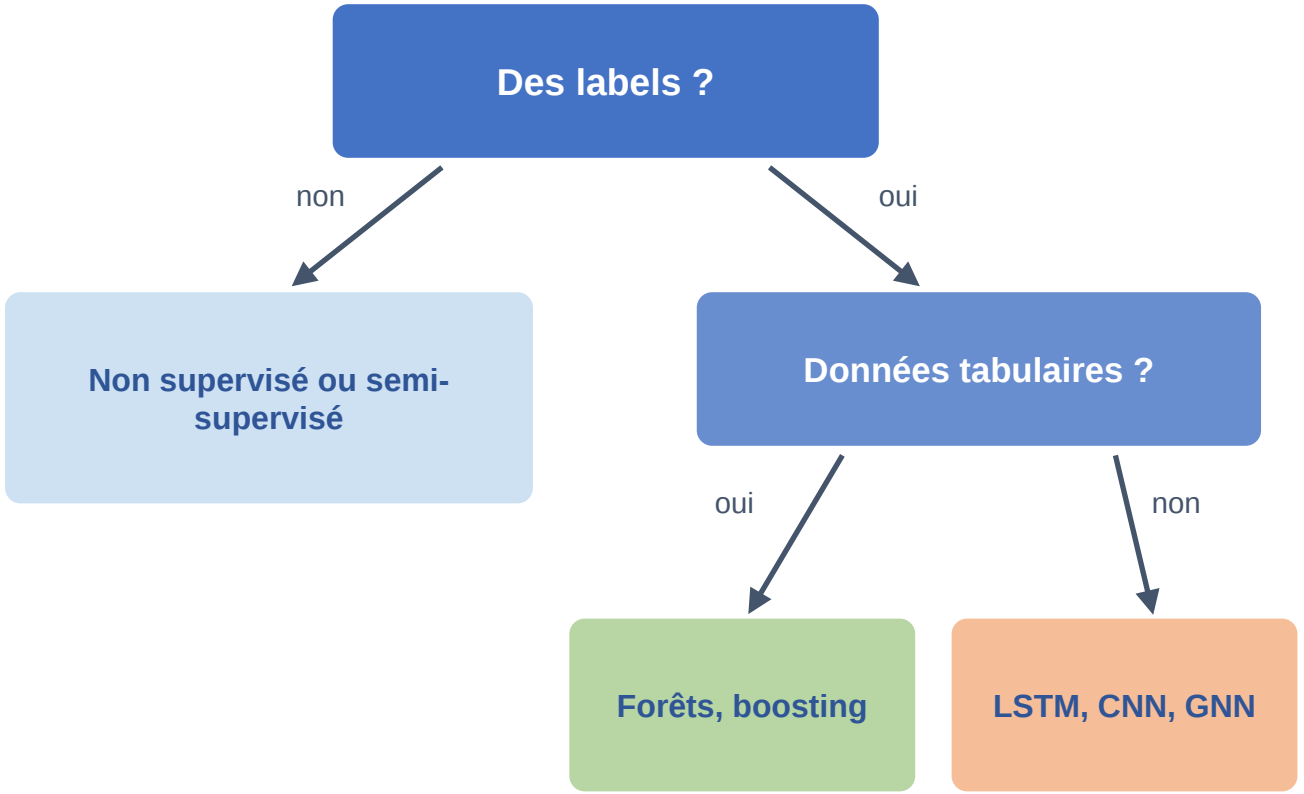
Entraîné sur du trafic normal uniquement

Deep learning : quel réseau pour quel problème ?

Architecture	Convient à	Exemple réseau
MLP	Features tabulaires	Classification de flux (rarement meilleur que les arbres)
CNN	Motifs locaux	Paquets bruts, séquences d'octets
LSTM / GRU	Séquences temporelles	Séquences de paquets, télémétrie
Transformer	Longues dépendances	Séquences de flux, logs
GNN	Graphes	Mouvement latéral, topologie

LLM : analyse de logs, assistants SOC/NOC, génération de configurations (voir l'ouverture).

Comment choisir un modèle ?



- **Latence** : inférence au débit ligne ?
- **Explicabilité** : l'opérateur doit-il comprendre ?
- **Volume** : assez de données pour le deep learning ?
- **Ressources** : CPU, mémoire, GPU disponibles

Message clé : un modèle simple sur de bonnes features bat souvent un modèle complexe sur de mauvaises.

Sur-apprentissage et jeu de test

- **Sur-apprentissage** (overfitting) : le modèle apprend les données d'entraînement par cœur, bruit compris
- **Symptôme** : excellents résultats à l'entraînement, mauvais sur de nouvelles données
- **Causes** : modèle trop complexe, peu de données
- **Parade** : toujours évaluer sur des données jamais vues

Entraînement (≈ 70 %)
ajuste les paramètres du modèle

Validation (≈ 15 %)
choisit le modèle et ses réglages

Test (≈ 15 %)
mesure finale, utilisée une seule fois

Évaluer un modèle : matrice de confusion

	Prédit : attaque	**Prédit : normal**
Réel : attaque	VP : attaque détectée	FN : attaque manquée
Réel : normal	FP : fausse alarme	VN : normal reconnu

Précision **VP / (VP + FP)**

Parmi les alertes, combien sont vraies ?

Rappel **VP / (VP + FN)**

Parmi les attaques, combien détectées ?

F1 **2·P·R / (P + R)**

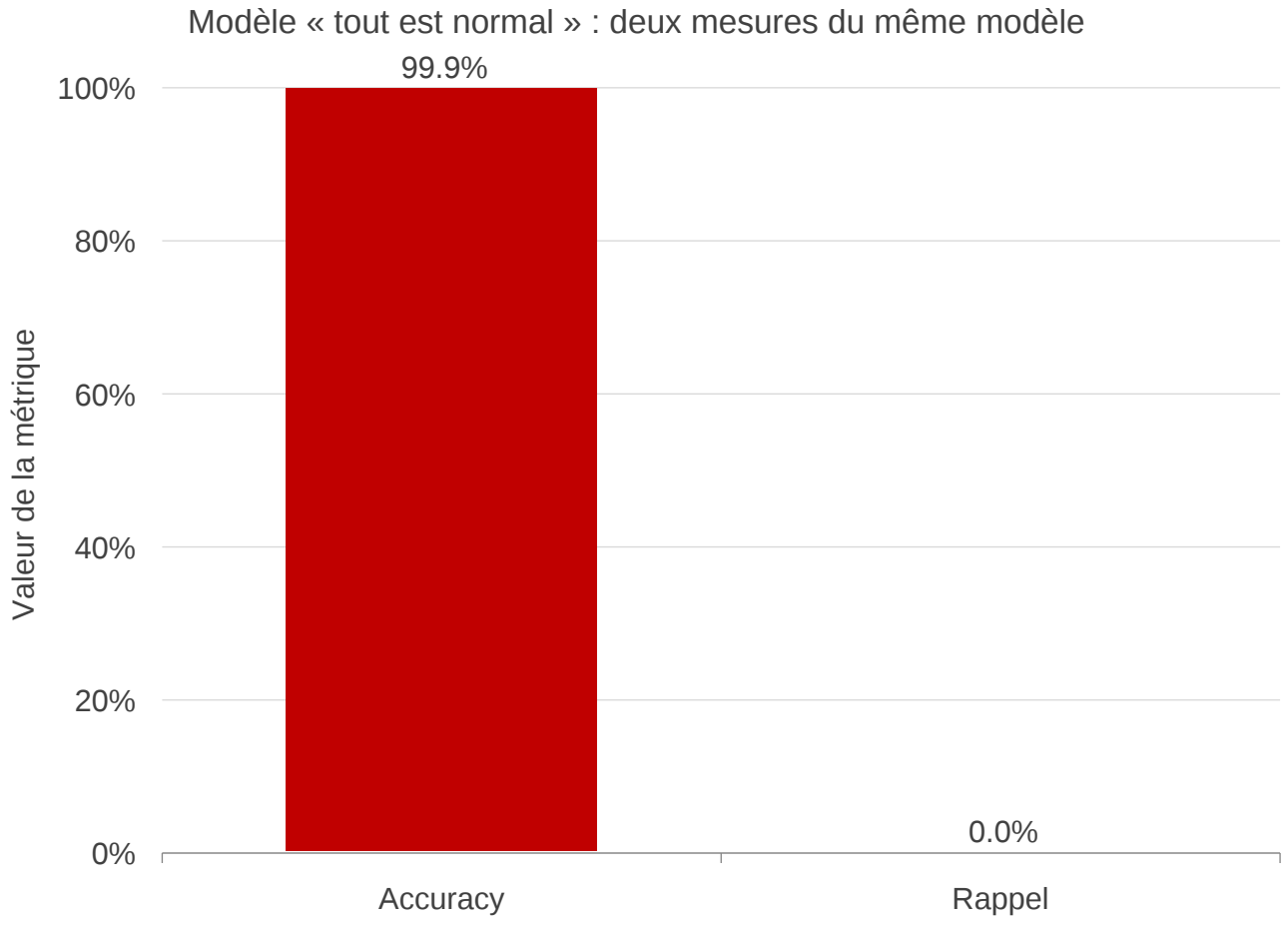
Compromis précision / rappel

Fausses alarmes **FP / (FP + VN)**

Part du normal signalé à tort

Ici « positif » = attaque. Le schéma vaut pour toute question oui / non : flux vidéo ou non, panne ou non, QoE dégradée ou non.

Le piège de l’accuracy



- **Situation** : 1 000 000 flux, dont 1 000 attaques (0,1 %)
- **Modèle « tout est normal »** : il ne détecte rien
- **Accuracy** = bonnes réponses / total = 999 000 / 1 000 000 = **99,9 %**
- **Rappel** = attaques détectées / attaques réelles = 0 / 1 000 = **0 %**
- **Conclusion** : une accuracy excellente peut cacher un modèle inutile

Deux modèles, une accuracy trompeuse

	Modèle A (détecteur réel)	Modèle B (« tout est normal »)
Attaques détectées	800 sur 1 000	0 sur 1 000
Fausses alarmes	2 000	0
Accuracy	99,78 %	99,9 %
Rappel	80 %	0 %
Précision	28,6 %	non définie (aucune alerte)

Le modèle B « gagne » sur l'accuracy et ne sert à rien : il faut regarder le rappel et la précision.

Le paradoxe du taux de base

Valeur	Signification
1 000 000	flux analysés
1 000	attaques réelles (0,1 %)
9 990	fausses alarmes à 1 % de fausses alarmes
≈ 9 %	précision, même avec 100 % de rappel

- Un détecteur « à 99 % » peut noyer les analystes : environ 9 alertes sur 10 sont fausses
- Quand l’événement est rare, la précision dépend surtout du taux de fausses alarmes

Découpage temporel et fuite de données

Découpage aléatoire : le test contient des flux très proches de ceux de l'entraînement

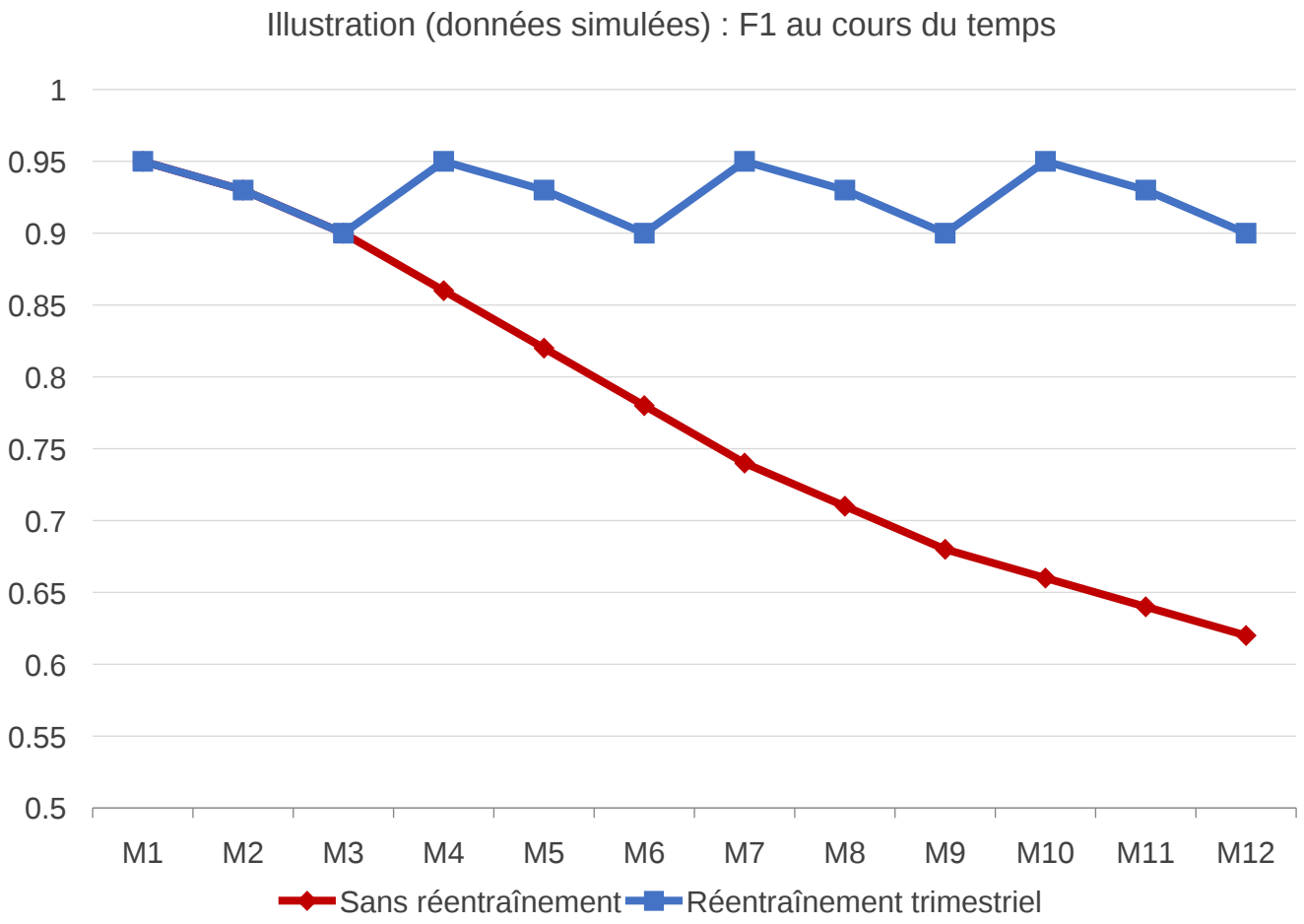


Découpage temporel : on entraîne sur le passé, on teste sur le futur



- **Fuite** : features calculées avec des informations du futur, ou même session des deux côtés
- **Groupes** : découper par hôte, par jour ou par capture

Dérive de concept



- **Causes** : nouvelles applications, QUIC et HTTP/3, mises à jour des lecteurs, nouvelles attaques
- **Détecter** : surveiller la distribution des features et les métriques
- **Réagir** : réentraînement planifié ou déclenché

Datasets : ce qu'ils contiennent, leurs biais

Dataset	Année	Contenu	Problèmes connus
KDD Cup 99	1999	Connexions TCP décrites par 41 features ; attaques DoS, sondes, R2L, U2R (trafic simulé)	Très redondant, trafic daté ; encore cité à tort
NSL-KDD	2009	Version nettoyée de KDD99 (doublons retirés)	Même trafic obsolète
UNSW-NB15	2015	Trafic généré en laboratoire ; 9 familles d'attaques ; 49 features	Génération synthétique, classes déséquilibrées
CIC-IDS2017	2017	Cinq jours de trafic (bénin, force brute, DoS, botnet, DDoS…) ; environ 80 features par flux	Erreurs d'étiquetage et artefacts qui gonflent les scores

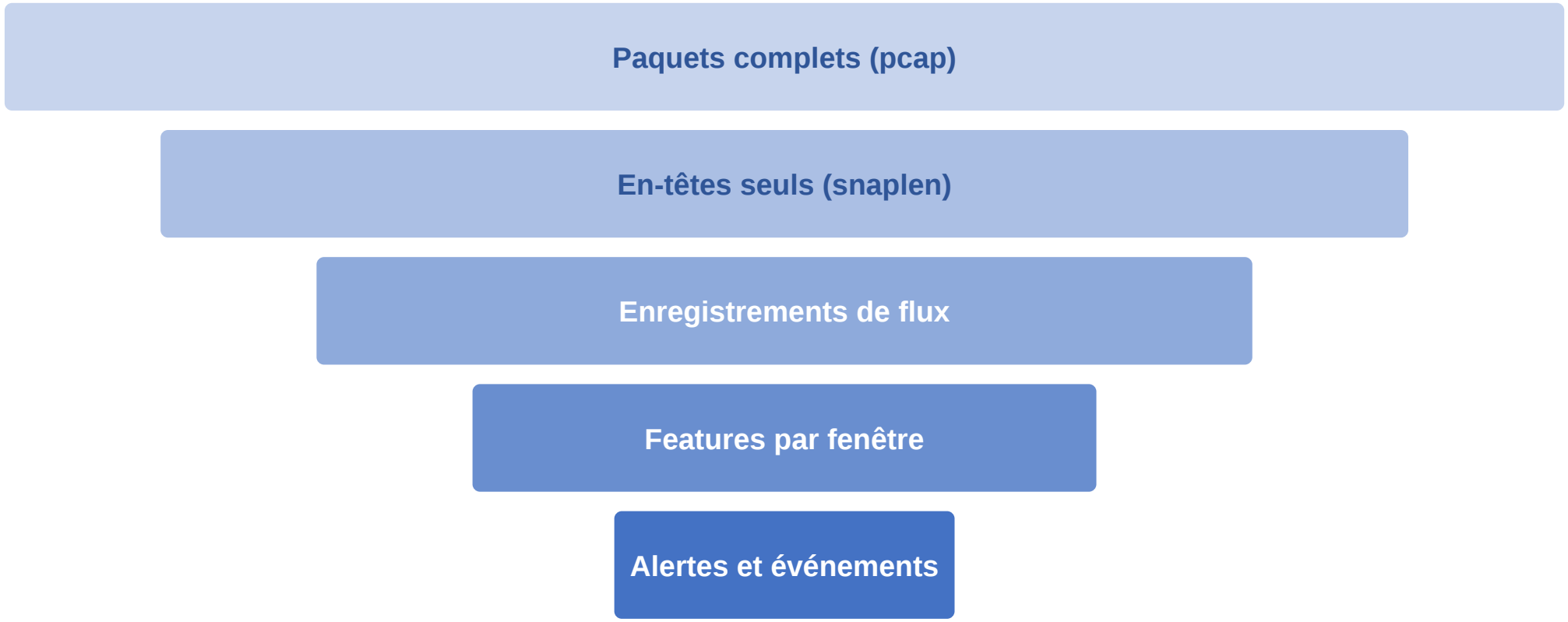
À lire : Sommer & Paxson (2010), Arp et al. (USENIX Security 2022), Engelen et al. (2021) sur CIC-IDS2017.

À retenir : l’IA pour les réseaux

- **Entrées** : la source de collecte impose la granularité : paquets, flux, séries, logs, graphes.
- **Sorties** : classe, score, valeur, série : le choix de la sortie définit la tâche et la métrique.
- **Évaluation** : précision et rappel, découpage temporel, dérive : la plupart des erreurs viennent de là.

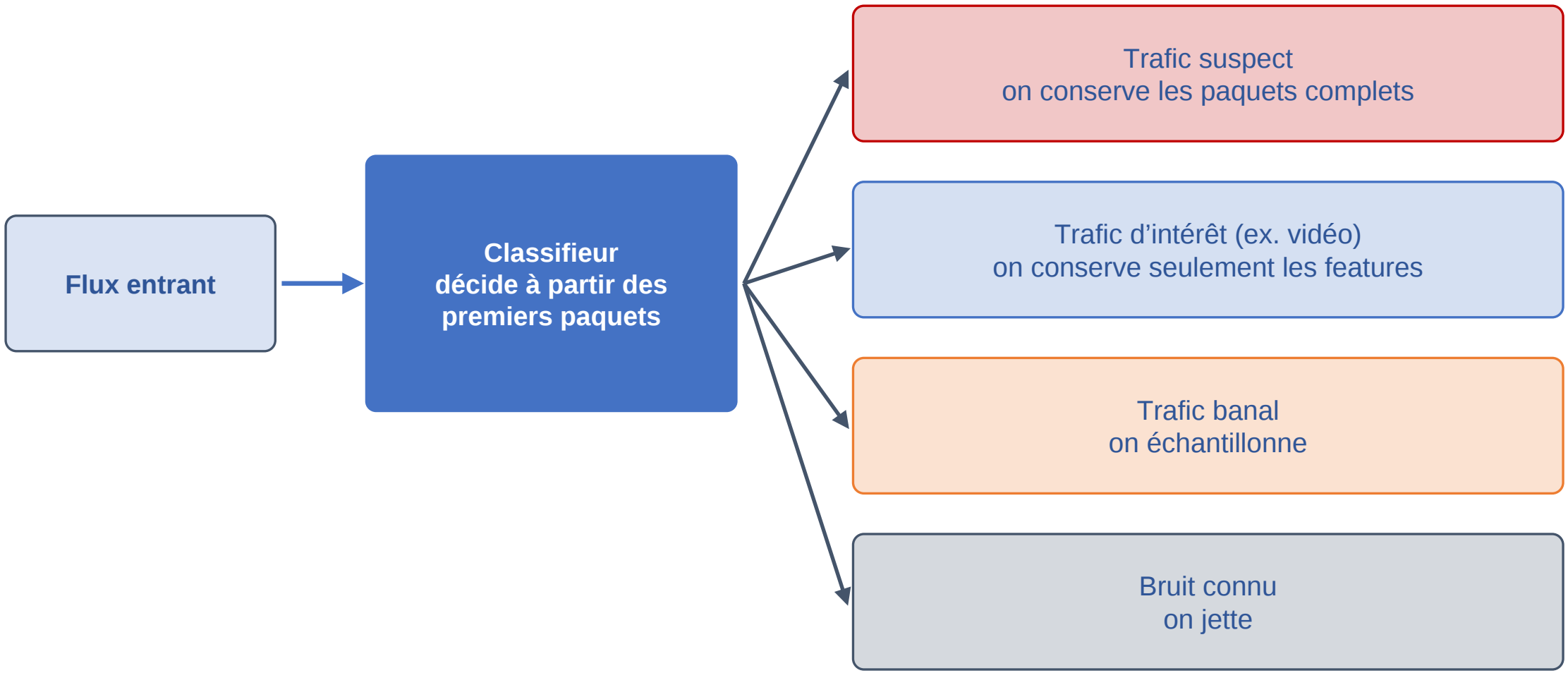
3. Classification : réduire le volume dès la collecte

On ne peut pas tout garder

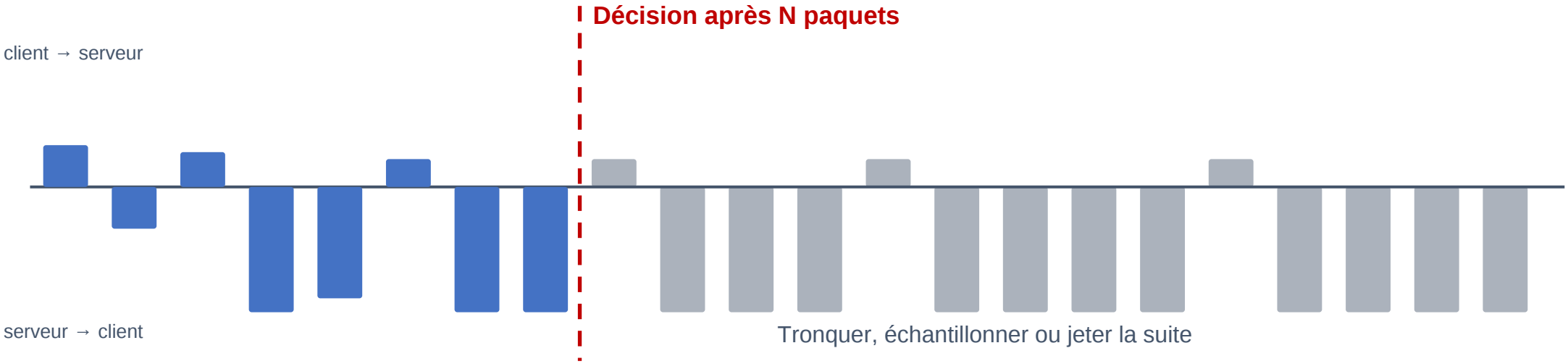


Chaque étape divise le volume de plusieurs ordres de grandeur, au prix d'une perte de détail

Classifier pour décider quoi conserver



Classification précoce sur les premiers paquets



- **Idée** : chaque application a un début d'échange caractéristique (poignée de main, premières requêtes), visible dans les tailles et le sens des premiers paquets
- **Résultat fondateur** : Bernaille, Teixeira et Salamatian (CoNEXT 2006) montrent que les tailles des 5 premiers paquets d'un flux TCP suffisent à identifier l'application
- **Compromis** : N petit = décision rapide ; N grand = plus précis

Que classe-t-on, et pourquoi est-ce possible ?

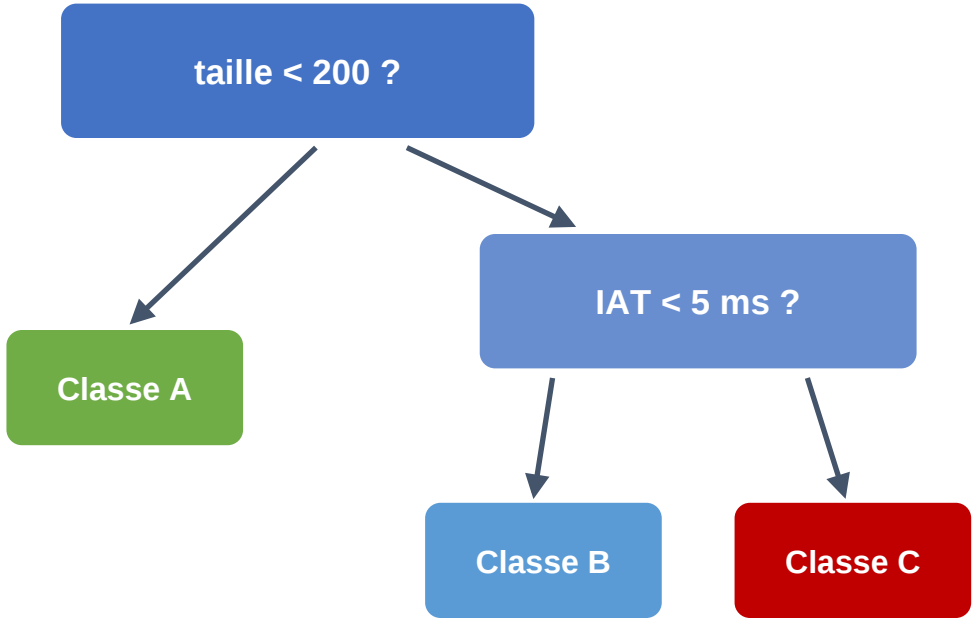
- **Pourquoi c’est possible** : chaque application ou comportement laisse une signature dans les tailles, les sens et les temps des paquets, même chiffrés
- **Application** : vidéo, web, VoIP, jeu. Sert par exemple à isoler le trafic vidéo avant l’inférence de QoE (partie 5)
- **Malveillant ou bénin** : pour garder les paquets complets des flux suspects (partie 4)
- **Priorité de conservation** : un score d’intérêt pour garder, résumer ou jeter selon la valeur de l’information

Où classifier ?

Emplacement	Débit supporté	Complexité du modèle	Atout et limite
Collecteur derrière le TAP	Moyen (CPU, DPDK)	Élevée (forêts, deep learning)	Flexible ; copie complète du trafic nécessaire
Noyau : XDP / eBPF	Élevé	Faible (arbres, règles)	Décision à la source ; contraintes du vérificateur
Plan de données : P4	Débit ligne	Très faible (arbres quantifiés)	Latence minimale ; matériel programmable, peu de mémoire

Exécuter un modèle dans le commutateur (P4)

P4 est un langage qui programme ce que fait un commutateur de chaque paquet. Objectif : décider à la vitesse du lien, sans envoyer le paquet à un serveur.



traduit en table de règles

Taille	IAT (ms)	Action
0–199	*	classe A
200–1500	0–4	classe B
200–1500	5 et +	classe C

- **Contraintes** : pas de flottants, peu de mémoire, pas de boucles
- **Taille** : un arbre de profondeur 4 a au plus 16 feuilles, soit quelques dizaines de règles (ordre de grandeur)
- **Référence** : IIsy (Xiong & Zilberman, HotNets 2019)

Le coût des erreurs de classification

	Décision : conserver	**Décision : jeter**
Réalité : suspect	Conservé à raison	Suspect jeté : angle mort définitif
Réalité : banal	Volume conservé pour rien	Économie réalisée

- Un **faux négatif est irréversible** : la donnée jetée n'existe plus
- Un faux positif coûte seulement du stockage
- On règle donc le seuil pour un **rappel élevé**
- Mesurer les deux sur des données récentes

Précision et rappel se lisent ici en volume conservé et en détections perdues.

Classer sur des métadonnées chiffrées

Indice	Ce qu'il révèle	Résiste au chiffrement ?
Tailles de paquets et sens	Type d'application, phases d'échange	Oui
Inter-arrivées, rafales	Rythme (segments vidéo, requêtes)	Oui
SNI / ALPN	Service, protocole (HTTP/2, HTTP/3)	Jusqu'à ECH
Empreinte du handshake (JA3, JA4)	Famille de client logiciel	Oui (ClientHello visible)
Charge utile	Contenu	Non

Contraintes de débit et de latence

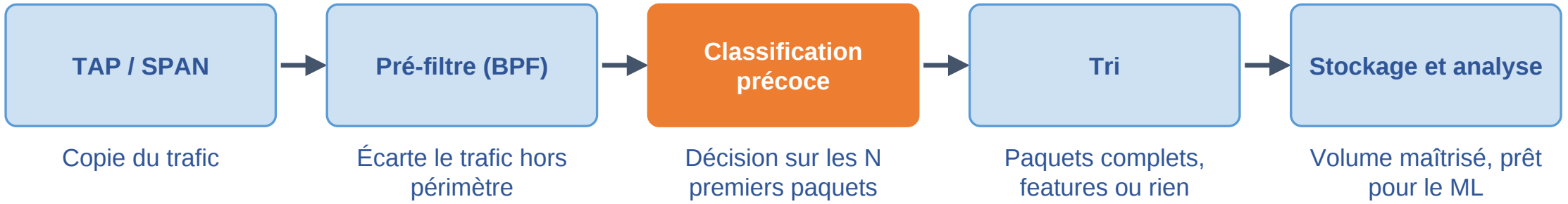
Question : combien de temps a-t-on pour traiter un paquet, au pire cas (paquets de 64 octets) ?

Débit du lien	Paquets par seconde	Temps disponible par paquet
10 Gb/s	14,88 millions	≈ 67 ns
100 Gb/s	148,8 millions	≈ 6,7 ns

- **Contrainte 1** : extraire les features et appliquer le modèle doit tenir dans ce temps
- **Contrainte 2** : un accès à la mémoire principale prend de l'ordre de 100 ns, déjà plus que le budget à 10 Gb/s
- **Conséquences** : décider par flux (premiers paquets) et non par paquet ; modèles très simples (arbres, règles) ; calcul au plus près de la carte (XDP, P4)

Pipeline complet de réduction

Du trafic brut au volume utile : chaque étape écarte ce qui n’est pas nécessaire.



On conserve des features par flux ou par fenêtre plutôt que des paquets, sauf pour les flux suspects.

À retenir : la classification

- **Pourquoi** : tout garder est impossible : le volume impose de décider tôt.
- **Comment** : classification précoce sur les premiers paquets, à la source (eBPF, P4).
- **À quel prix** : un faux négatif est une donnée perdue ; le chiffrement limite les indices.

4. Cas d’usage 1 : la détection

Deux cas d’usage : détection et performances

Détection d’anomalies (sécurité)

- Question : ce trafic est-il malveillant ou anormal ?
- Outils : SIEM, IDS, NDR, SOAR
- Difficulté : faux positifs, attaquant adaptatif
- Public : analystes du SOC

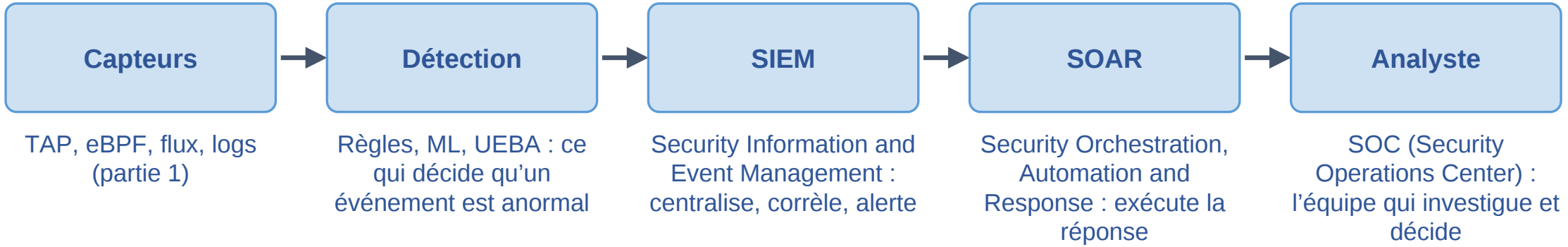
Inférence de QoE (performance)

- Question : quelle qualité l’abonné perçoit-il ?
- Outils : sondes du FAI, télémétrie client
- Difficulté : trafic chiffré, généralisation
- Public : opérateurs, FAI

Même pipeline, objectifs différents : détection dans cette partie, performances dans la suivante.

Le pipeline de détection de bout en bout

Cinq maillons, du réseau à l'analyste. Dans la suite, un repère « Où on est » indique le maillon traité.

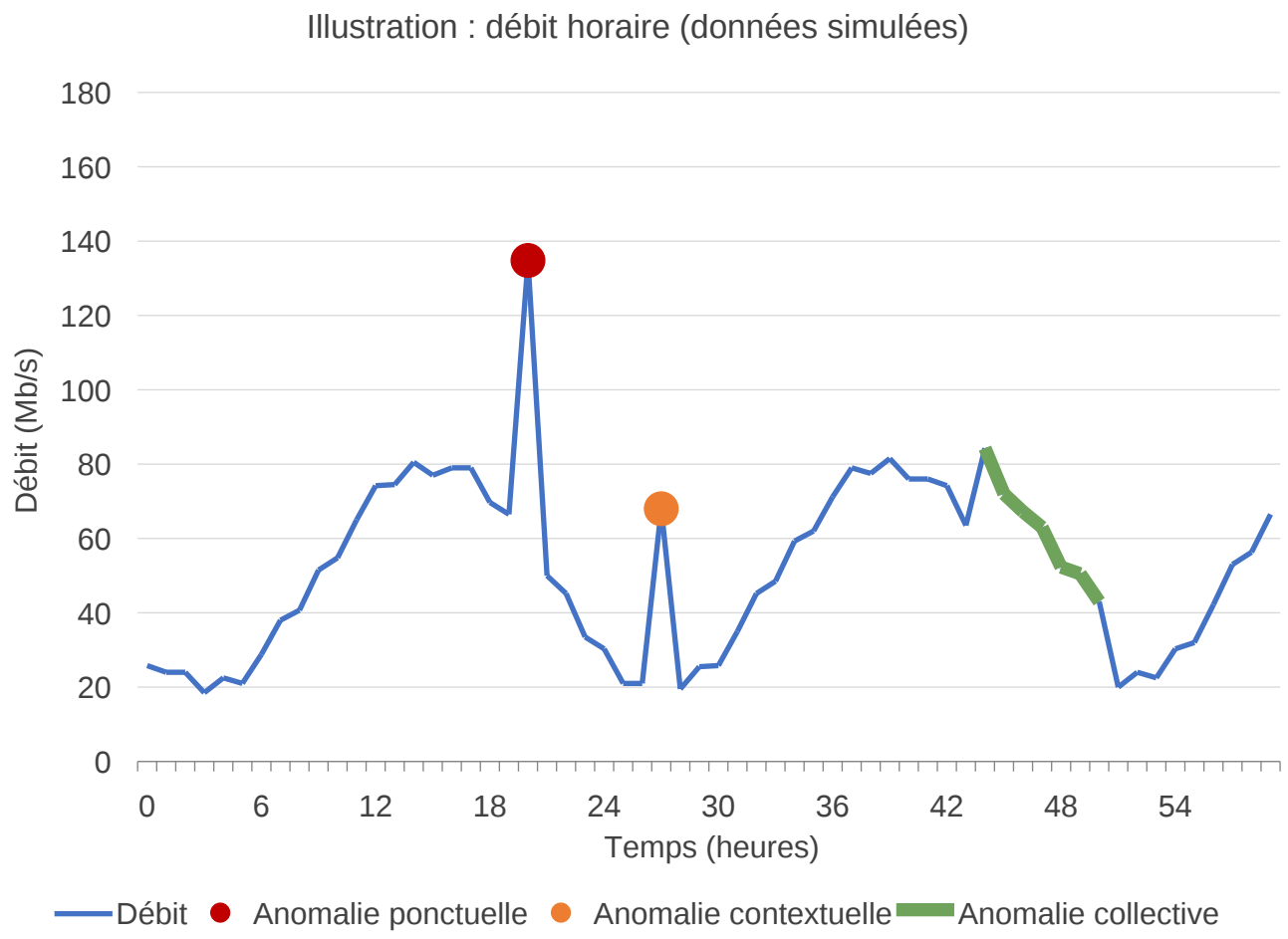


UEBA : User and Entity Behavior Analytics, l'analyse du comportement des utilisateurs et des machines.

Qu’est-ce qu’une anomalie ?

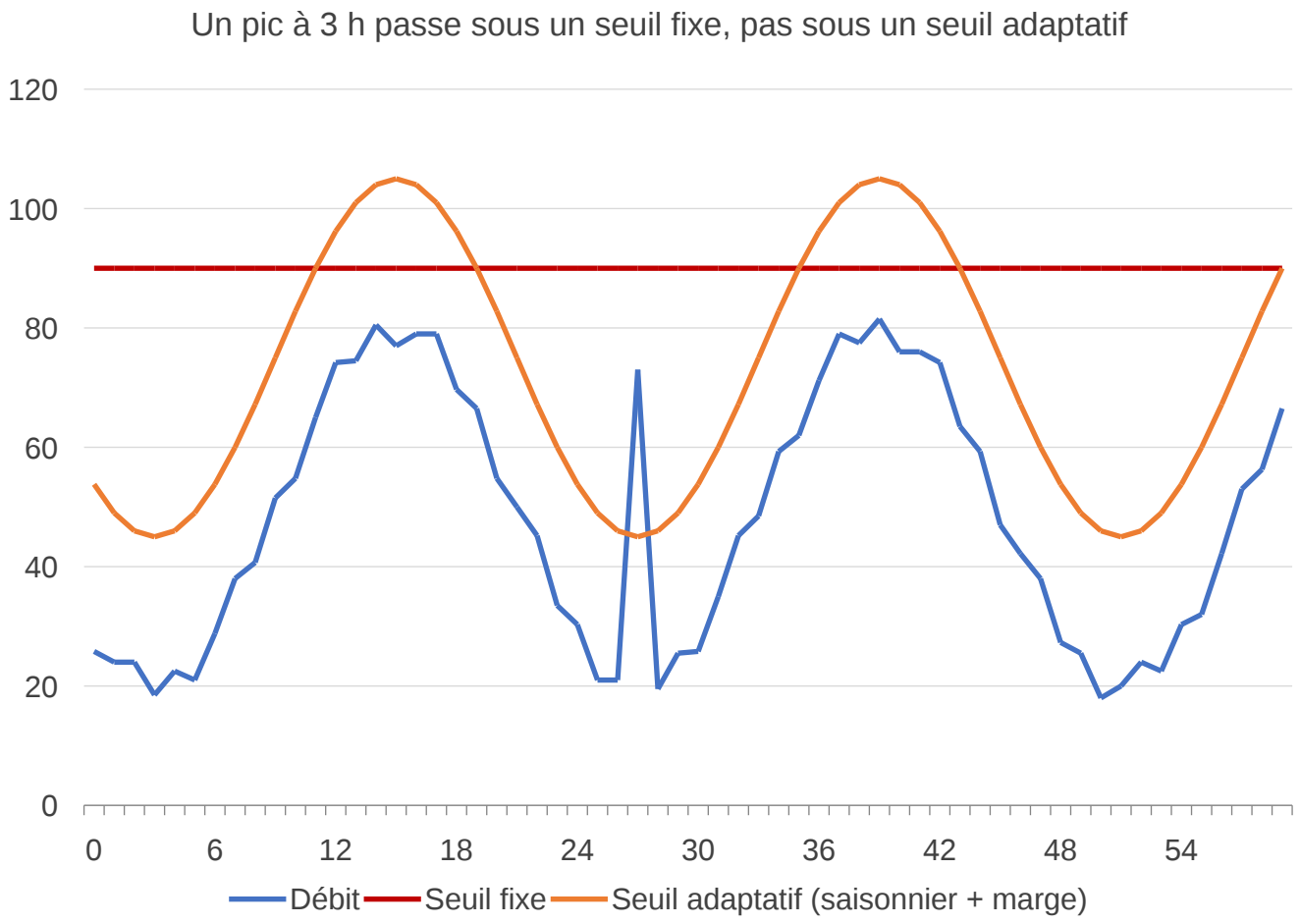
- **Panne** : lien coupé, équipement défaillant : le débit s’effondre.
- **Mauvaise configuration** : boucle, route erronée, ACL trop large : trafic inattendu.
- **Attaque** : DDoS, scan, exfiltration, mouvement latéral, force brute.
- **Événement légitime** : afflux de visiteurs, sauvegarde, mise à jour : inhabituel mais bénin.

Trois types d’anomalies



- **Ponctuelle** (rouge) : une valeur extrême isolée, ici un pic vers l’heure 20
- **Contextuelle** (orange) : normale ailleurs, anormale ici : un pic à 3 h du matin (heure 27)
- **Collective** (vert) : une suite de valeurs anormales ensemble, ici de l’heure 44 à 50 (exfiltration lente)

Baseline : le seuil statistique



- **Seuil fixe** : simple, mais aveugle à la saisonnalité
- **Adaptatif** : moyenne glissante, EWMA, percentiles par heure
- **Règle d'or** : tout modèle ML doit battre cette baseline

Règles de corrélation dans un SIEM

Où on est : Capteurs | Détection | **SIEM** | SOAR | Analyste

```
title: SSH brute force
logsource:
  product: linux
  service: auth
detection:
  selection:
    event.category: authentication
    event.outcome: failure
  timeframe: 1m
  condition: selection | count() by source.ip >= 5
level: medium
```

- **Exemple** : 5 échecs de connexion en 1 minute depuis une même IP
- **Syntaxe** inspirée de Sigma, convertible vers Splunk, Elastic, Sentinel
- **Source** : logs d'authentification normalisés
- **Action** : alerte, ticket, playbook

Règles et signatures : forces et limites

Forces

- Lisibles et auditables
- Peu de faux positifs si bien écrites
- Rapides à exécuter (Snort, Suricata)
- Rattachables aux techniques connues (MITRE ATT&CK)

Limites

- Aveugles aux attaques inconnues ou obfusquées
- Maintenance continue des règles
- Contournables si l'attaquant connaît le seuil
- Explosion du nombre de règles

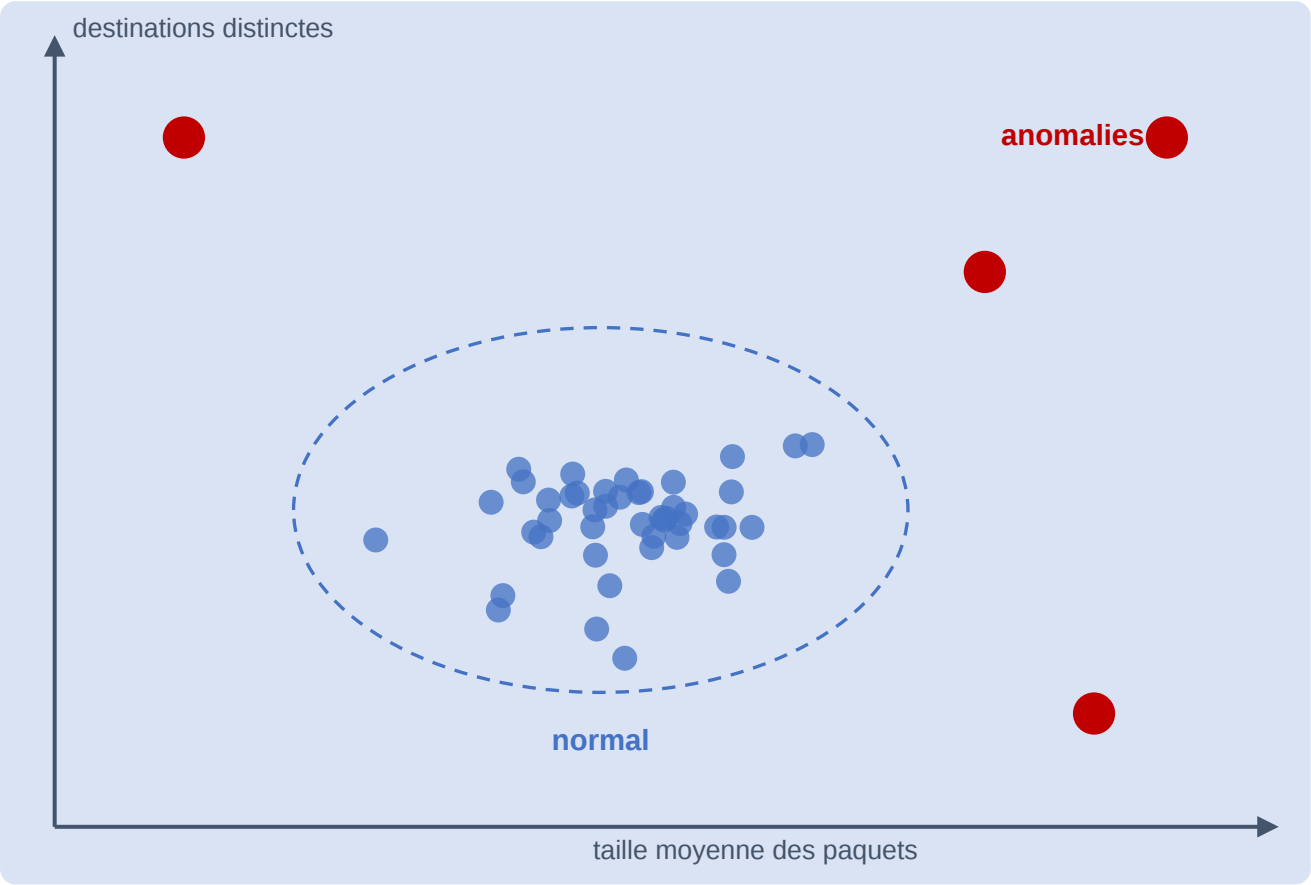
Signature = ce qu'on sait déjà. Anomalie = ce qui sort de l'ordinaire. Les deux se complètent.

ML supervisé : un IDS appris

Où on est : Capteurs | **Détection** | SIEM | SOAR | Analyste

- **Entrée** : features de flux (tailles, durées, flags)
- **Sortie** : bénin ou type d'attaque (DoS, scan, force brute…)
- **Modèles** : forêts aléatoires, gradient boosting, réseaux de neurones
- **Atout** : très bon sur les attaques présentes dans les données
- **Limites** : labels coûteux, faible généralisation à de nouvelles attaques et à d'autres réseaux
- **Résultat à relativiser** : des scores de 99 % sur CIC-IDS2017 reflètent souvent des artefacts du dataset plutôt qu'une détection robuste.

Non supervisé : isoler les points atypiques



- **Isolation Forest** : score élevé = isolé en peu de coupes
- **Distance, densité** : LOF, DBSCAN
- **Seuil** sur le score : compromis faux positifs / rappel
- **Piège** : « inhabituel » ne veut pas dire « malveillant »

Séries temporelles : prédire, puis comparer

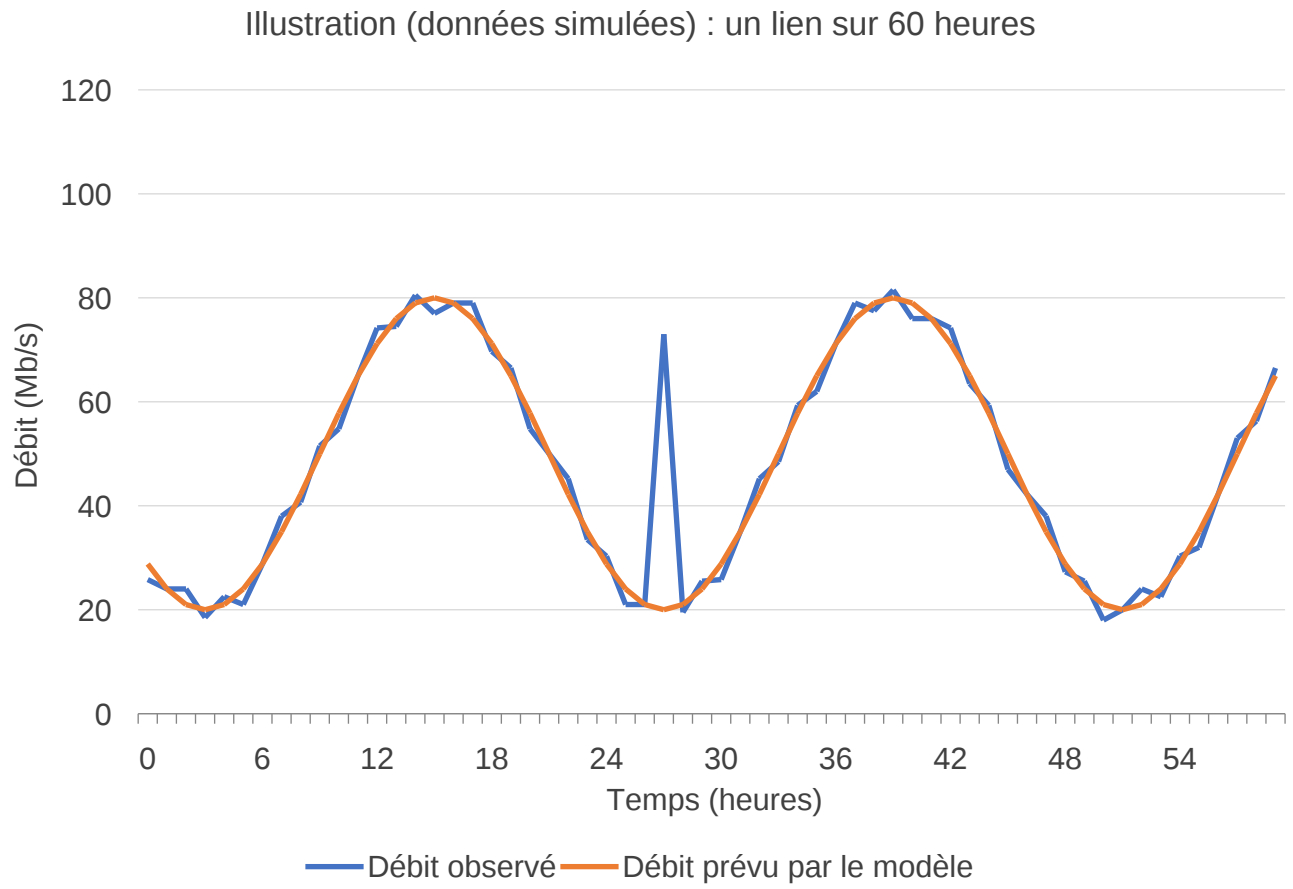
Où on est : Capteurs | **Détection** | SIEM | SOAR | Analyste

1. **Série observée** : débit, latence, compteurs par intervalle
2. **Modèle** : tendance et saisonnalité (STL, Holt-Winters) ou LSTM
3. **Prévision** : valeur attendue et intervalle de confiance
4. **Résidu** : écart entre l'observé et l'attendu
5. **Alerte** : résidu au-delà d'un seuil, sur plusieurs points

Adapté à la télémétrie (SNMP, gNMI). Géré dans les SIEM par des modules de ML (ex. Elastic ML).

Exemple : prévoir le débit, puis comparer

Où on est : Capteurs | **Détection** | SIEM | SOAR | Analyste



- **Série** : le débit horaire d'un lien
- **Modèle** : il a appris le cycle de 24 h et prévoit la valeur attendue (courbe orange)
- **Résidu** = observé − prévu : proche de 0 en temps normal
- **Heure 27** (3 h du matin) : observé ≈ 73, prévu ≈ 20, résidu ≈ +53 : anomalie

UEBA : définition et différence avec les séries temporelles

Séries temporelles (une technique)

- Objet : une mesure, ex. le débit d'un lien
- Question : cette valeur est-elle normale pour ce moment ?
- Exemple : un pic de débit à 3 h du matin
- Outils : prévision, résidu

UEBA (une approche)

- User and Entity Behavior Analytics
- Objet : le profil d'une entité (utilisateur, machine) sur plusieurs dimensions
- Question : cette entité se comporte-t-elle comme d'habitude, ou comme ses pairs ?
- Exemple : un utilisateur télécharge 50 Go à une heure inhabituelle

Pas concurrents : l'UEBA s'appuie sur des techniques comme les séries temporelles ou le ML.

UEBA : analyser le comportement des entités

Où on est : Capteurs | **Détection** | SIEM | SOAR | Analyste

1. **Événements** : authentifications, accès, flux
2. **Profils** : comportement habituel par utilisateur, machine, groupe
3. **Écarts** : heure, volume ou destination inhabituels, comparés au groupe
4. **Score de risque** : agrégé par entité, remonté à l'analyste

Intégré aux SIEM (ex. Microsoft Sentinel, Elastic). Exemple : un poste qui envoie 50 Go vers une destination jamais vue.

Faux positifs et fatigue des alertes

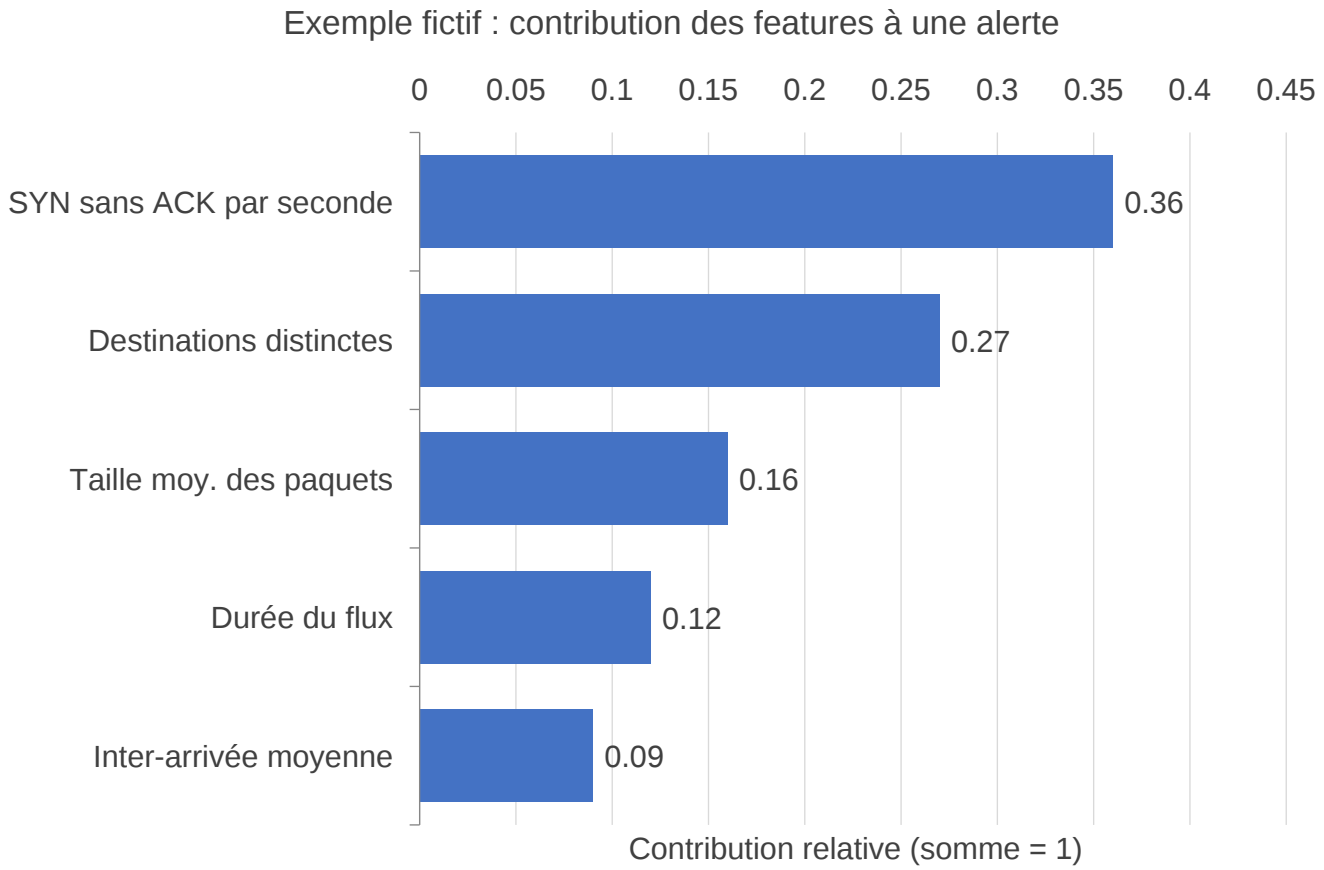
Où on est : Capteurs | Détection | SIEM | SOAR | **Analyste**

- **Alertes brutes** : règles et ML produisent beaucoup de bruit
  - Exemple : un outil d'inventaire interne qui scanne le réseau déclenche 200 alertes de scan
- **Regroupement** : dédoublonner et corréler par entité et par période
  - Exemple : ces 200 alertes deviennent 1 incident
- **Priorisation** : score de risque, criticité des actifs, contexte
  - Exemple : une alerte sur le serveur de paie passe avant celle d'un poste de test
- **Analyste SOC** : traite quelques incidents, avec explication
- **Retour** : l'analyste marque « faux positif » et le modèle en tient compte

Chaque fausse alerte coûte du temps d'analyste ; trop de bruit, et les vraies alertes sont ignorées.

Explicabilité : pourquoi cette alerte ?

Où on est : Capteurs | Détection | SIEM | SOAR | **Analyste**



- **Situation** : le modèle signale un flux comme « scan de ports »
- **Graphique** : la part de chaque feature dans cette décision (méthode de type SHAP)
- **Lecture** : les SYN restés sans réponse (0,36) pèsent le plus : le flux envoie beaucoup de demandes de connexion sans suite, typique d'un scan
- **Valeurs fictives**, données à titre d'illustration

Robustesse face à un attaquant adaptatif

Où on est : Capteurs | **Détection** | SIEM | SOAR | Analyste

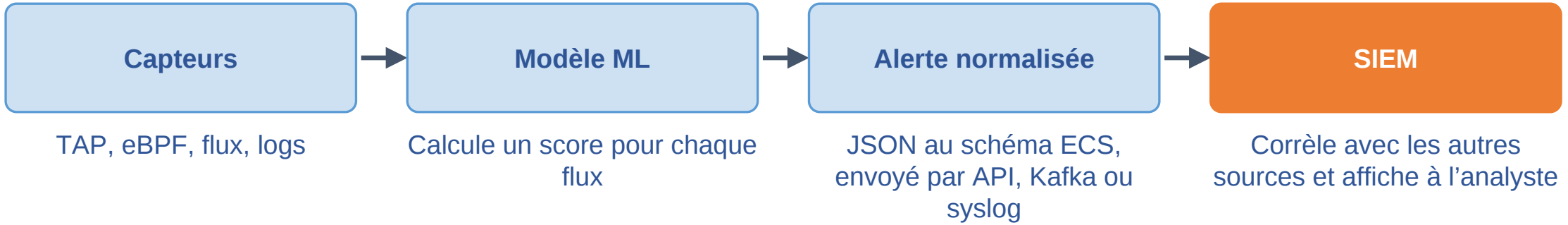
- **Évasion** : L'attaquant imite le trafic normal ou ralentit pour rester sous le seuil.
- **Empoisonnement** : il pollue les données d'entraînement pour décaler le « normal ».
- **Exemples adverses** : perturbations ciblées des features pour franchir la frontière de décision.

Défenses : réentraînement surveillé, features difficiles à manipuler, défense en profondeur (règles + ML). Cf. Arp et al. 2022.

Intégration : les alertes remontent dans le SIEM

Où on est : Capteurs | Détection | **SIEM** | SOAR | Analyste

Le modèle ML est un composant extérieur au SIEM. L'intégrer, c'est lui faire envoyer ses alertes au SIEM, pour qu'elles s'affichent avec les autres.



Le SIEM reste la console unique : le modèle est une source d'alertes parmi d'autres.

SOAR : automatiser la réponse

Où on est : Capteurs | Détection | SIEM | **SOAR** | Analyste

- **SOAR** : Security Orchestration, Automation and Response. Il exécute des scénarios (playbooks) déclenchés par une alerte du SIEM
- **Exemple** : alerte « scan depuis l'IP X » → vérifier la réputation de X → bloquer X sur le pare-feu → ouvrir un ticket



Risque : un faux positif déclenche une action réelle (blocage d'un client légitime). Garde-fous : validation humaine, seuils, rollback.

Comparer les approches de détection

Approche	Forces	Limites	Quand l'utiliser
Seuil statistique	Simple, explicable	Saisonnalité, bruit	Baseline, supervision
Règles SIEM	Auditables, précises	Maintenance, attaques inconnues	Menaces connues, conformité
ML supervisé	Précis sur le connu	Labels, généralisation	Catalogue d'attaques étiqueté
ML non supervisé, UEBA	Détecte l'inconnu	Faux positifs, explicabilité	Comportements, premier tri

5. Cas d’usage 2 : inférence des performances réseau

Objectif : déduire la QoE à partir de la QoS

1. **Ce que veut l'opérateur** : connaître la qualité vécue par ses abonnés (la QoE)
2. **Ce qu'il mesure** : la qualité du réseau (la QoS : débit, latence, pertes)
3. **Ce qu'on va faire** : apprendre un modèle qui déduit la QoE à partir du trafic observé
4. **Rôle de Netflix** : une plateforme qui mesure la QoE directement, donc qui fournit la vérité terrain
5. **Rôle du FAI** : déduire la QoE sans voir le contenu chiffré

QoS ou QoE ?

QoS (réseau)

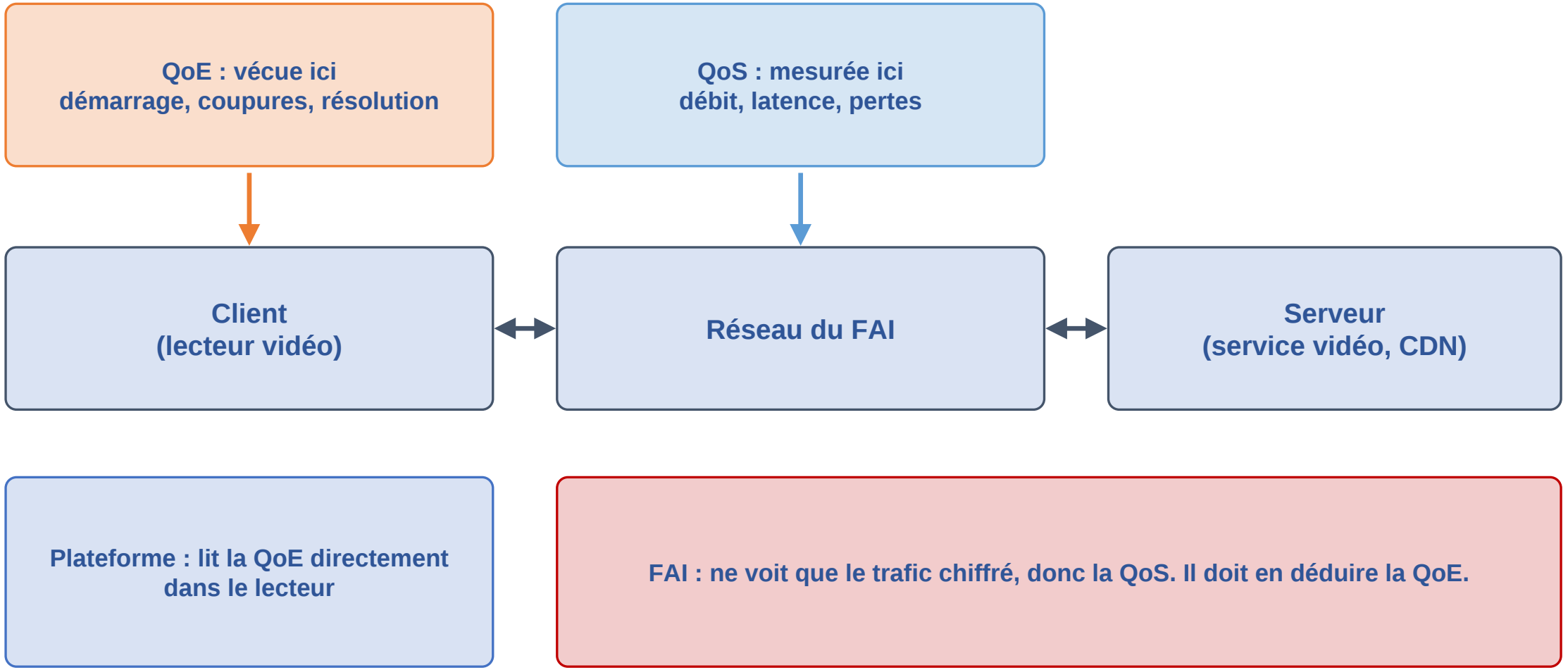
- Débit, latence, gigue, pertes
- Mesurée dans le réseau
- Objective, facile à collecter
- Ne dit pas ce que l’utilisateur ressent

QoE (utilisateur)

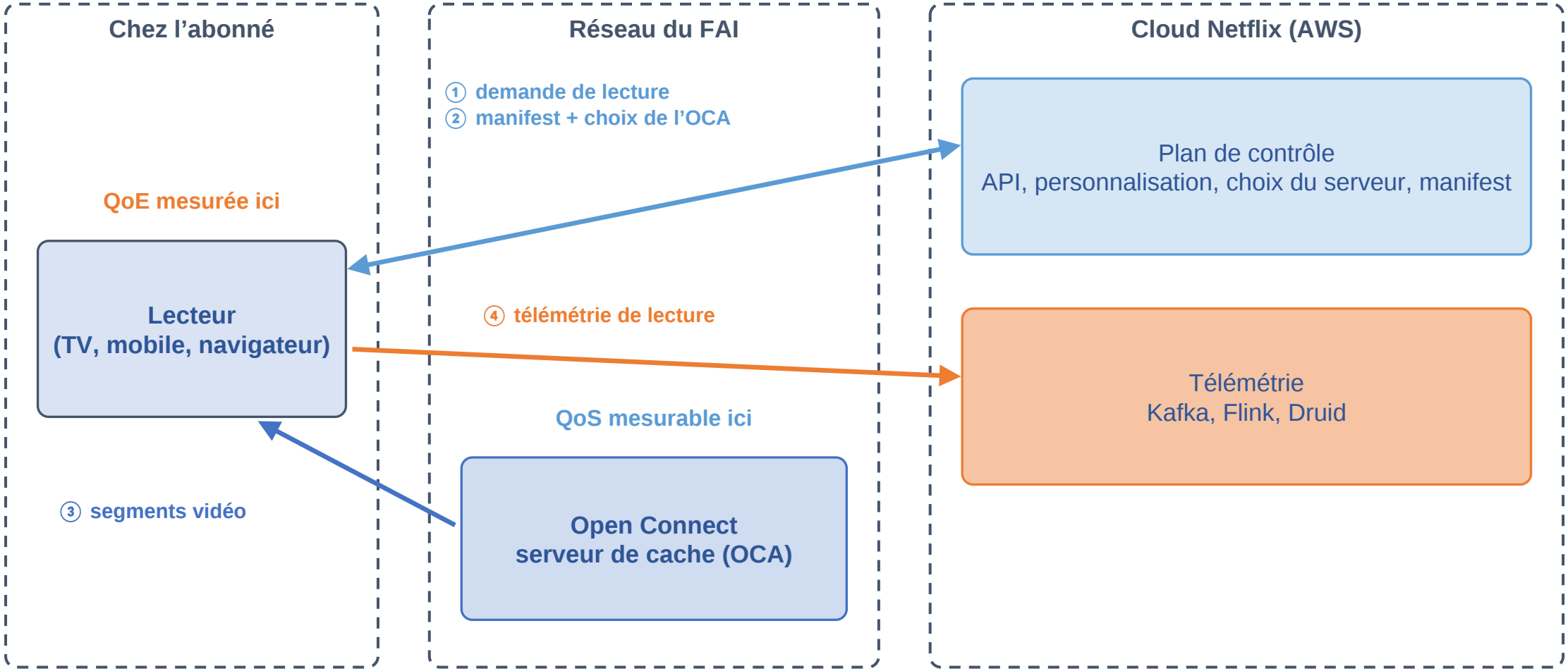
- Délai de démarrage, interruptions (rebuffering)
- Résolution et changements de qualité
- Mesures objectives (VMAF) ou subjectives (MOS)
- Corrélée à l’abandon et à la satisfaction

L’opérateur veut connaître la QoE, mais le réseau ne mesure que la QoS : il faut l’inférer.

Deux points de vue sur la même session vidéo



Architecture Netflix : vue d'ensemble



Source : publications publiques de Netflix (simplifié).

Open Connect : le CDN de Netflix

- **Open Connect** : le réseau de diffusion (CDN) de Netflix, c’est-à-dire des serveurs de cache qui contiennent les vidéos
- **Où** : installés dans les réseaux des FAI ou aux points d’échange, au plus près des abonnés
- **Pourquoi c’est important ici** : la vidéo arrive depuis le réseau du FAI, chiffrée. Le FAI voit passer le trafic, pas la qualité de lecture

Fil conducteur : Netflix mesure la QoE directement ; le FAI doit l’inférer à partir de la QoS.

Source : Netflix Open Connect (documentation et publications publiques).

La télémétrie de lecture côté client

- **Démarrage** : délai avant la première image.
- **Interruptions** : fréquence et durée du rebuffering.
- **Qualité** : débit vidéo, résolution, changements de qualité.
- **Erreurs et contexte** : codes d’erreur, appareil, version, pays, FAI.

Ces événements sont émis par les lecteurs en temps réel : c’est la vérité terrain de la QoE.

Source : Netflix Technology Blog (observabilité de la lecture avec Druid).

Le pipeline temps réel de Netflix

Objectif : repérer en quelques secondes une dégradation de la QoE.



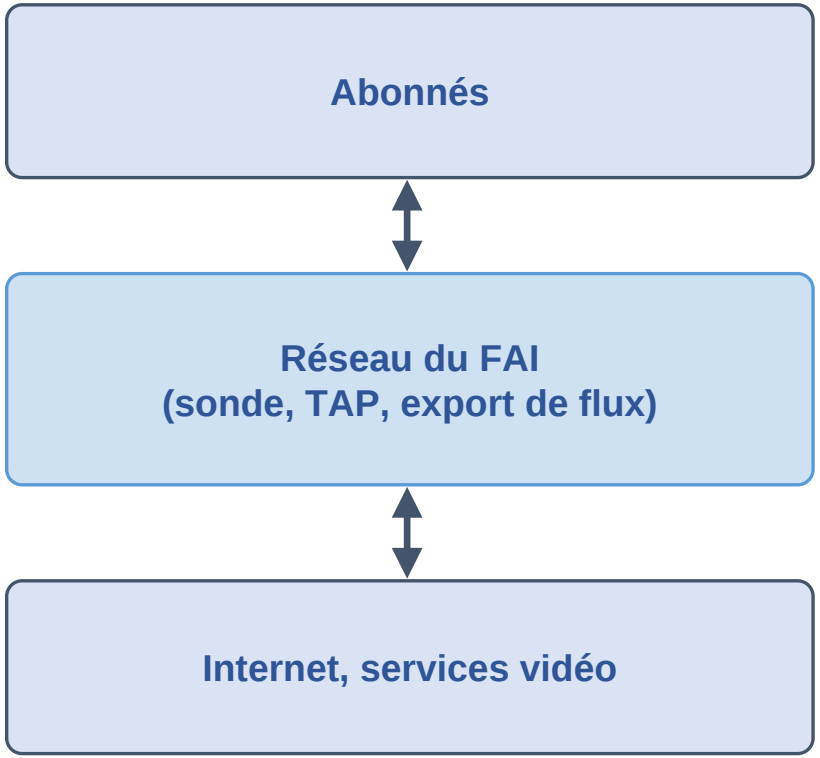
Lecteurs	Kafka	Flink (Keystone)	Druid	Tableaux de bord et alertes
Chaque lecteur envoie des événements de lecture : démarrage, interruptions, qualité	File de messages : reçoit les événements en continu et les met en attente	Traitement de flux : filtre, enrichit et route les événements vers les bons destinataires	Base d'analyse temps réel : répond en quelques secondes sur des milliards d'événements	Détection de régressions par appareil, version, région

Ordre de grandeur : jusqu'à 38 millions d'événements par seconde lors des plus grands événements en direct.

Sources : Netflix Technology Blog (Druid ; infrastructure de streaming en direct, 2025). Autres briques : Atlas (métriques), Mantis (flux opérationnels).

Qu’est-ce qu’un FAI ?

- **FAI** : fournisseur d’accès à Internet (en anglais ISP), l’opérateur qui relie les abonnés à Internet (Orange, Free, SFR, Bouygues…)
- **Il possède le réseau d’accès** : tout le trafic de ses abonnés passe chez lui
- **Il peut y placer des sondes** (TAP, export de flux) pour observer le trafic
- **Mais** le trafic est chiffré : il voit les paquets, pas le contenu ni la qualité de lecture

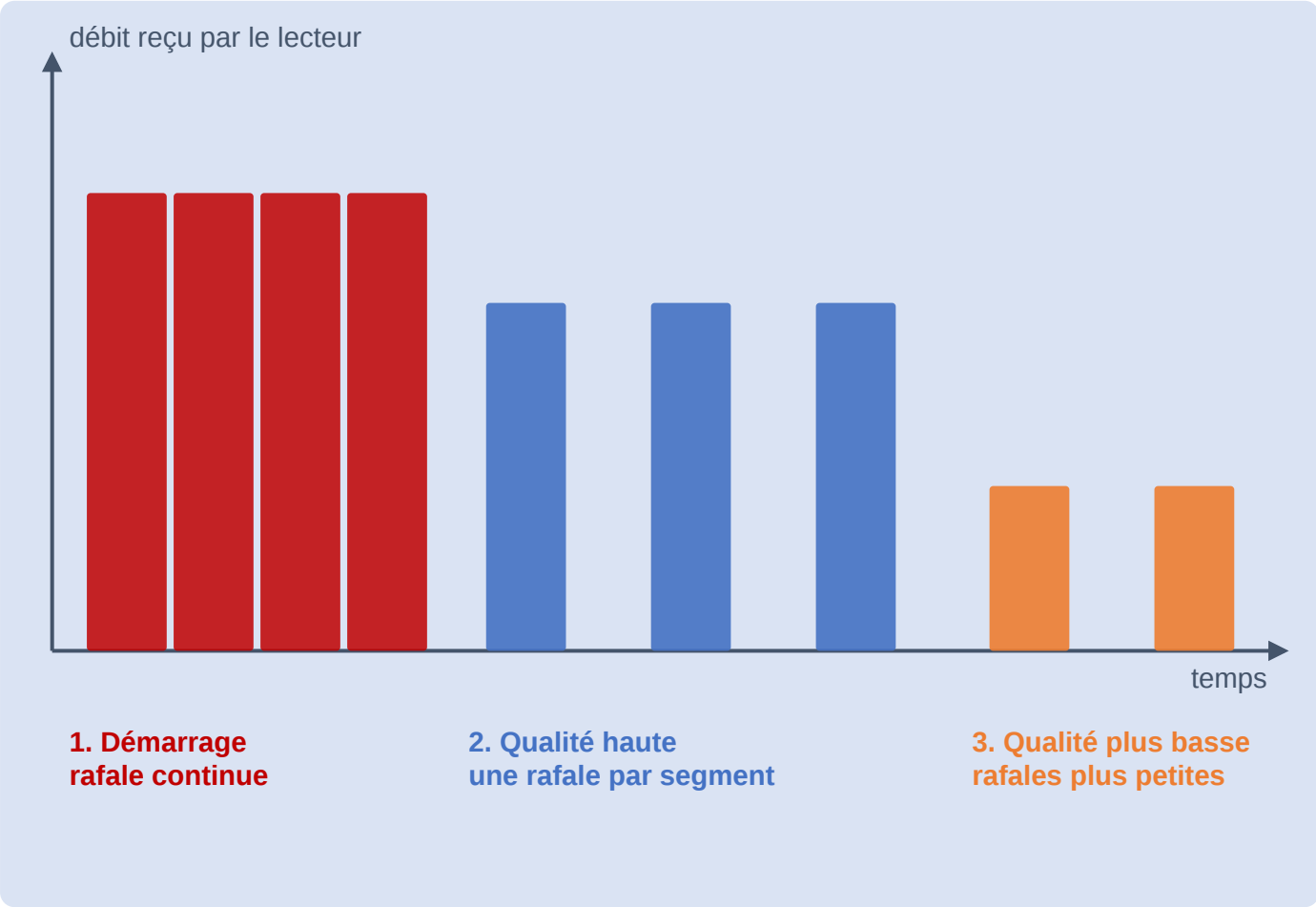


Le point de vue du FAI : inférer sans voir

1. **Trafic chiffré** : segments HTTPS ou QUIC : contenu opaque
2. **Sonde** : TAP ou IPFIX côté FAI
3. **Features** : tailles et rythme des segments, débit par fenêtre
4. **Modèle** : classification et régression supervisées
5. **QoE estimée** : résolution, interruptions, démarrage par abonné

Même collecte que la partie 1, même pipeline que la partie 4 : seule la cible change.

Streaming adaptatif : DASH et HLS



Exemple schématique d'une session, vu depuis le réseau

- La vidéo est découpée en **segments** de quelques secondes (en direct chez Netflix : 2 s)
- Le lecteur choisit la qualité de chaque segment (**ABR**) selon le débit mesuré et son buffer
- La **hauteur** d'une rafale reflète la taille du segment, donc la qualité
- L'**espacement** des rafales reflète l'état du buffer
- Ce sont les indices de la slide suivante

Ce que le trafic révèle de la QoE

Indice dans le trafic (QoS)	Ce qu'il reflète	Cible (QoE)
Taille des segments	Débit vidéo choisi, donc résolution	Résolution
Délai entre requêtes	État du buffer	Interruptions
Rafale initiale	Phase de démarrage	Délai de démarrage
Débit par fenêtre	Capacité disponible	Changements de qualité

Ces indices survivent au chiffrement : ils sont dans les tailles et dans les temps.

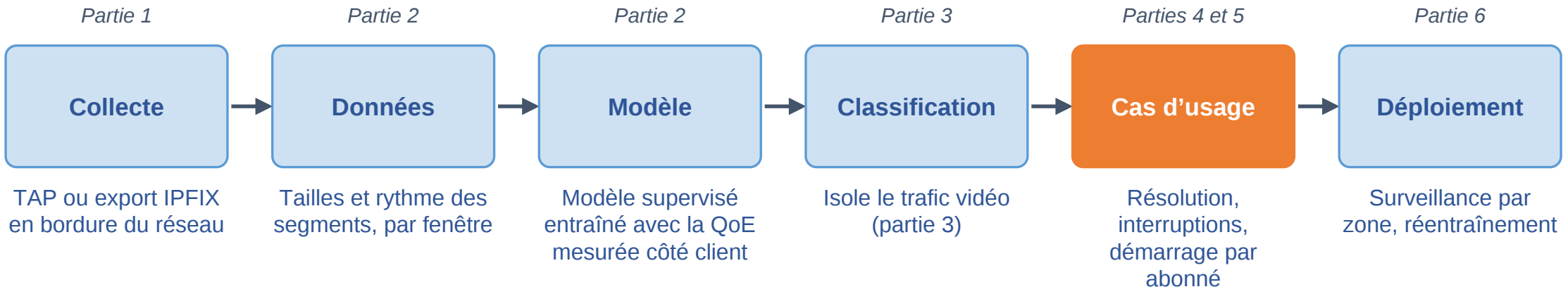
Cibles, modèles et travaux de référence

Travail	Cible	Approche
Mazhar & Shafiq, INFOCOM 2018	QoE temps réel sur HTTPS et QUIC	Features réseau, modèles supervisés
Bronzino et al., 2019	Qualité vidéo sur trafic chiffré, retour de déploiement	Modèles pratiques, évalués en conditions réelles
Requet (Gutterman et al., MMSys 2019)	Interruptions et résolution, YouTube chiffré, en temps réel	Apprentissage supervisé, labels par instrumentation du navigateur

Modèles typiques : forêts aléatoires, gradient boosting. Cibles : classification (résolution), régression (délai).

Le pipeline QoE du FAI, relié aux parties 1 à 3

Le pipeline du début du cours, appliqué à la QoE d'un FAI.



La classification précoce évite de traiter tout le trafic : seuls les flux vidéo alimentent le modèle QoE.

Limites de l’inférence de QoE

- **Généralisation** : un modèle appris sur un service ou un lecteur marche mal sur un autre.
- **Évolution** : nouveaux codecs, algorithmes d’adaptation, HTTP/3 : le trafic change.
- **Multiplexage** : QUIC et HTTP/2 mélangent plusieurs flux : les segments sont plus difficiles à isoler.
- **Vérité terrain** : labels obtenus par instrumentation de clients de test : coûteux, peu représentatifs.

Mesure directe ou inférence ?

	Mesure côté client (plateforme)	Inférence côté réseau (FAI)
Qui	Netflix, YouTube…	Opérateurs, FAI
Données	Télémétrie du lecteur	Trafic chiffré (tailles, temps)
Précision	Exacte	Estimée (erreurs possibles)
Couverture	Tous les clients du service	Tous les services, tous les abonnés
Accès	Contrôle du lecteur	Aucun accès au client

Complémentaires : la plateforme voit la lecture, le FAI voit le réseau.

Détection et QoE : le même pipeline

Étape	Détection d'anomalies	Inférence de QoE
Collecte	Logs, flux, paquets	Flux, paquets (trafic chiffré)
Entrées	Features par flux ou fenêtre, logs	Tailles et rythme des segments
Sortie	Score ou alerte	Résolution, interruptions, délai
Labels	Rares, parfois circulaires	Télémétrie client, instrumentation
Risque principal	Faux positifs, attaquant adaptatif	Généralisation, dérive

Supervision de performance ou sécurité : les méthodes sont transversales.

6. Déploiement et exploitation

Du notebook à la production

1. **Notebook** : exploration et prototype
2. **Pipeline** : features reproductibles, tests
3. **Entraînement** : versionner données et modèles
4. **Service** : inférence en ligne ou par lots
5. **Supervision** : qualité, dérive, latence

Contrainte réseau : l'inférence doit suivre le débit du trafic.

Où placer l’intelligence ?

Option	Avantage	Limite	Exemple
Centralisée (cloud, datacenter)	Modèles riches, vue globale	Latence, volume à remonter	SIEM, NDR
En périphérie (edge, hôte)	Décision locale, volume réduit	Ressources limitées	eBPF, sondes
Plan de données (switch)	Débit ligne, latence minimale	Modèles très simples	P4, SmartNIC

MLOps réseau : surveiller et réentraîner

1. **Surveiller** : distribution des features, métriques
2. **Détecter la dérive** : seuils sur les écarts
3. **Collecter** : nouvelles données étiquetées
4. **Réentraîner** : valider sur une période récente
5. **Déployer** : canari, retour arrière possible

Boucle continue : le trafic évolue, donc le modèle aussi.

7. Conclusion et ouverture

Les LLM pour l’exploitation et la sécurité

- **Analyse de logs** : résumer, expliquer, trier des événements ; requêtes en langage naturel sur le SIEM.
- **Copilotes SOC et NOC** : assistants intégrés aux SIEM pour guider l’investigation et proposer des réponses.
- **Configurations** : génération et revue de configurations : à valider avant tout déploiement.

Limites : hallucinations, confidentialité des données, latence, coût. L’humain reste dans la boucle.

Vers les réseaux autonomes

1. **Intention** : L'opérateur exprime un objectif (intent-based networking)
2. **Observer** : télémétrie et collecte continues
3. **Analyser** : ML : détection, prévision, diagnostic
4. **Décider** : planification sous contraintes
5. **Agir** : configuration automatique, boucle fermée

Vision des « self-driving networks » (Feamster & Rexford, 2018) : encore largement prospective.

Les limites qui restent

- **Confiance** : faut-il laisser un modèle agir seul ? Explicabilité et validation sont indispensables.
- **Responsabilité** : qui répond d’une coupure ou d’un blocage décidé par un algorithme ?
- **Sécurité des modèles** : évasion, empoisonnement, fuite de données : le modèle est lui-même une surface d’attaque.

Points à retenir

- **Collecter** : hôte, lien, équipement, application : chaque point de vue a ses angles morts.
- **Réduire** : classer tôt pour ne garder que l'utile ; un faux négatif est une donnée perdue.
- **Modéliser** : entrées, sorties, modèle : commencer simple, évaluer correctement.
- **Détecter** : seuil, règle, ML : le SIEM est la console, le SOAR l'exécutant.
- **Inférer** : la QoE se mesure côté client et s'infère côté réseau.
- **Exploiter** : surveiller la dérive, réentraîner, garder l'humain dans la boucle.

Références (1/2) : ML et évaluation

- **Sommer & Paxson**, Outside the Closed World: On Using Machine Learning for Network Intrusion Detection, IEEE S&P 2010
- **Arp et al.**, Dos and Don’ts of Machine Learning in Computer Security, USENIX Security 2022
- **Engelen, Rimmer, Joosen**, Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study, 2021
- **Boutaba et al.**, A comprehensive survey on machine learning for networking, J. Internet Services and Applications, 2018
- **Bernaille et al.**, Early Application Identification, CoNEXT 2006
- **Xiong & Zilberman**, Do Switches Dream of Machine Learning?, HotNets 2019

Références (2/2) : QoE, Netflix, eBPF, autonomie

- **Mazhar & Shafiq**, Real-time Video Quality of Experience Monitoring for HTTPS and QUIC, INFOCOM 2018
- **Bronzino et al.**, Inferring Streaming Video Quality from Encrypted Traffic: Practical Models and Deployment Experience, 2019
- **Gutterman et al.**, Requet: Real-Time QoE Detection for Encrypted YouTube Traffic, MMSys 2019
- **Netflix Technology Blog**, How Netflix uses Druid for real-time insights to ensure a high-quality experience ; infrastructure de streaming en direct (2025)
- **Feamster & Rexford**, Why (and How) Networks Should Run Themselves, ANRW 2018
- **Gregg**, BPF Performance Tools, Addison-Wesley, 2019